Magistrate Wizard architecture diagrams

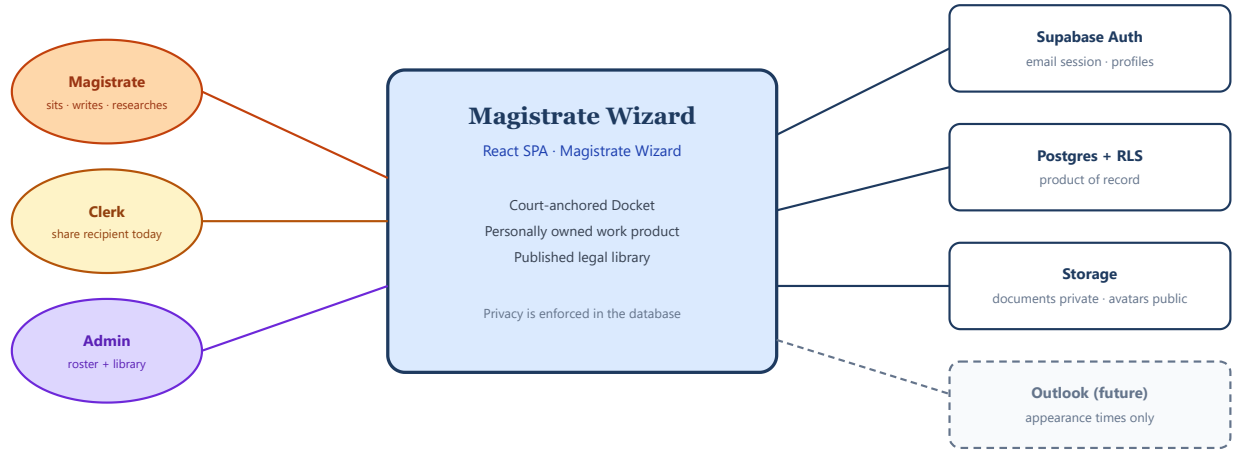
C4 context, entity-relationship diagrams, access control, user journeys, and sequence workflows. Authority: Final Architecture Specification Revision 3 and live migrations.

Magistrate Wizard
Architecture as implemented (migrations 0001–0067)
Court-anchored Docket · no admin bypass into private judicial content
13 August 2026

Hero diagrams

System context

Magistrate Wizard is one web app. Supabase is the only backend. Outlook is future.

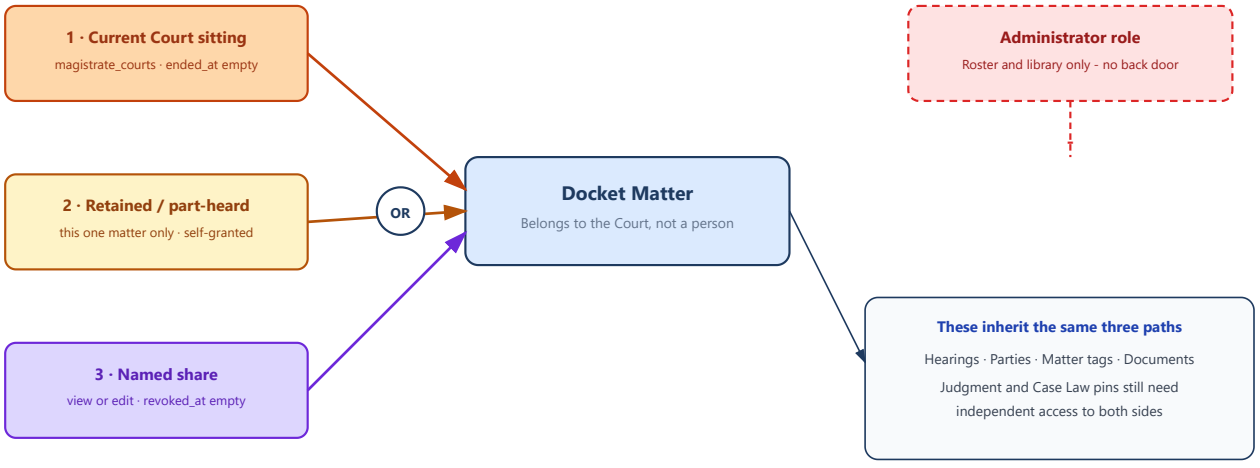


No Edge Functions in production today. Search and ingestion RPCs run inside Postgres.

System context — people, the app, and Supabase

How someone opens a Docket Matter

Three lawful paths. Any one is enough. Administrator is not a fourth path.

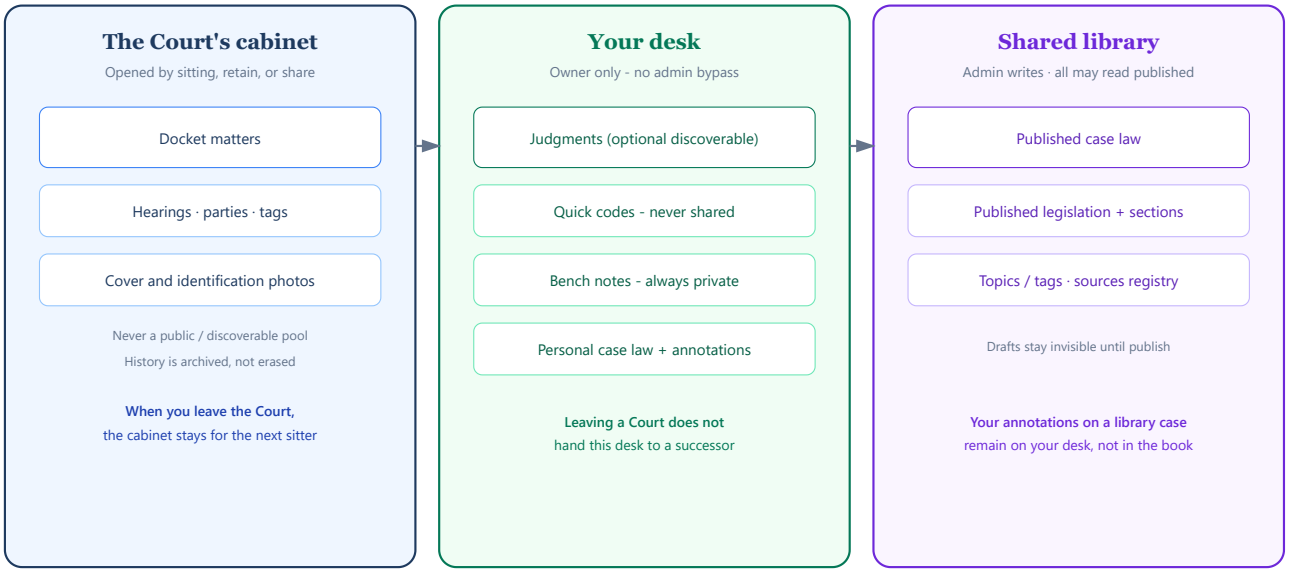


Magistrate Wizard · Court-Anchored Docket · can_access_court OR has_retained_assignment OR has_docket_share

Three paths onto a Docket Matter

Three cabinets, three lock types

A pin between cabinets never copies the keys.

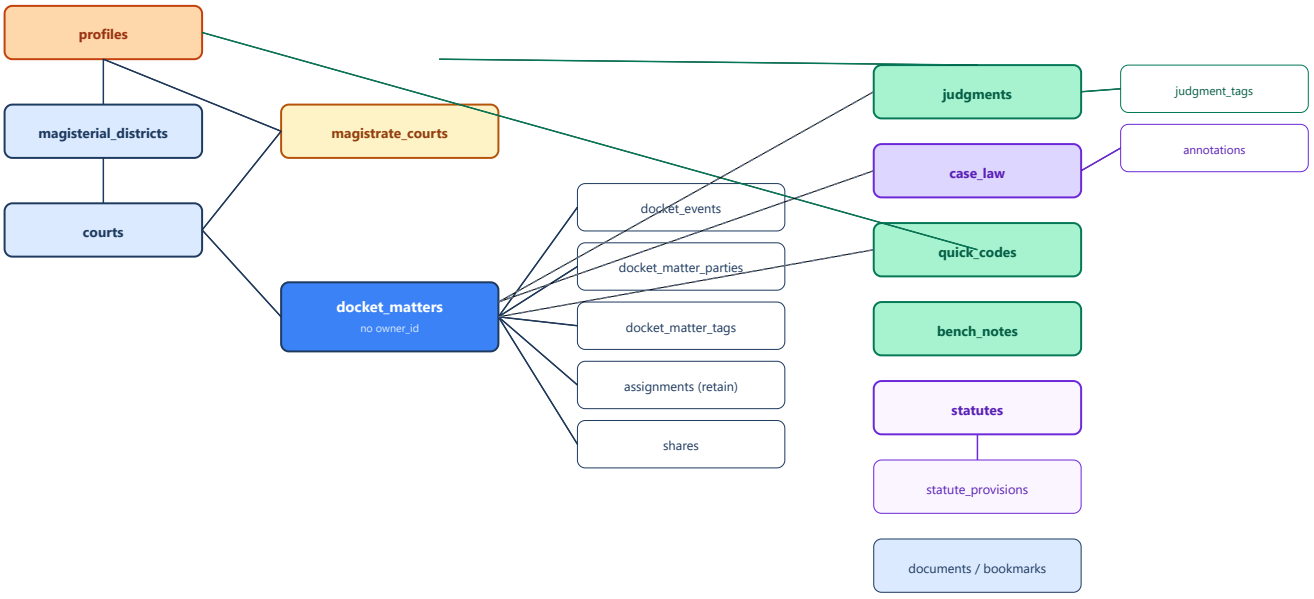


Dashed idea: a link is a pin, not a key · BOTH sides must already be readable

Court cabinet, personal desk, shared library

Entity map - who relates to whom

Boxes are tables. Crow's-feet mean "many". Dashed lines are association-only (a pin, not a key).

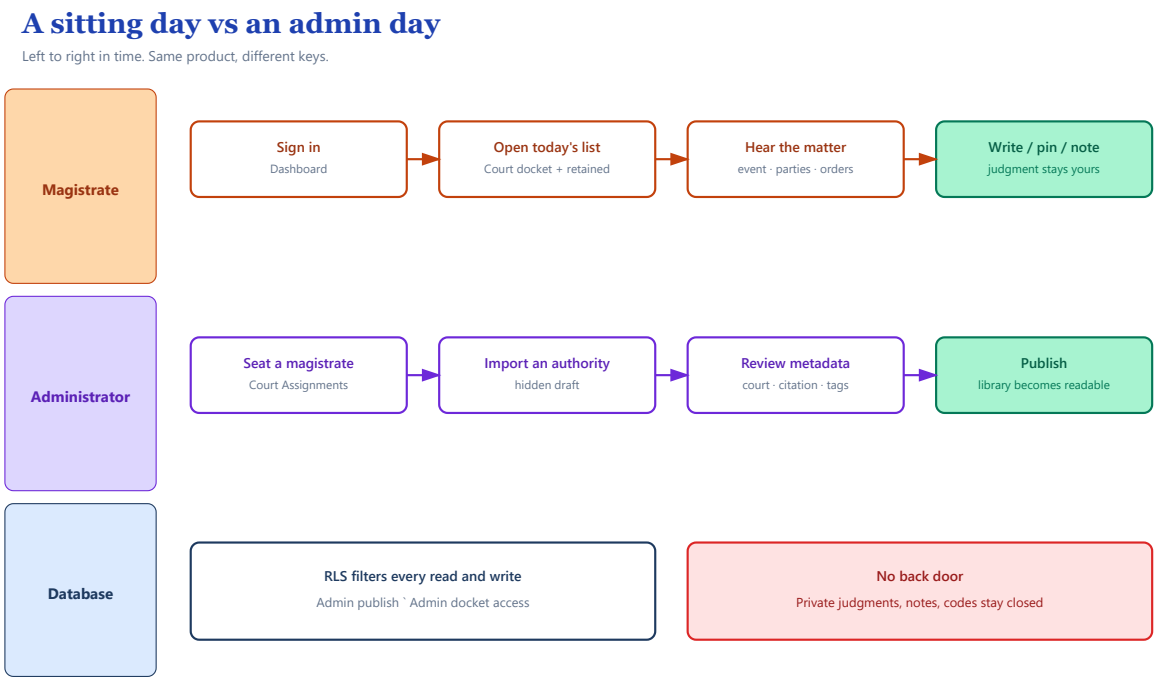


Solid line parent / child or ownership

Dashed line association table - BOTH sides already readable, never confers access

Legacy cases / case_parties omitted - unused by new development. legal_authority_courts is a separate "reported court" list, not courts.

Entity map



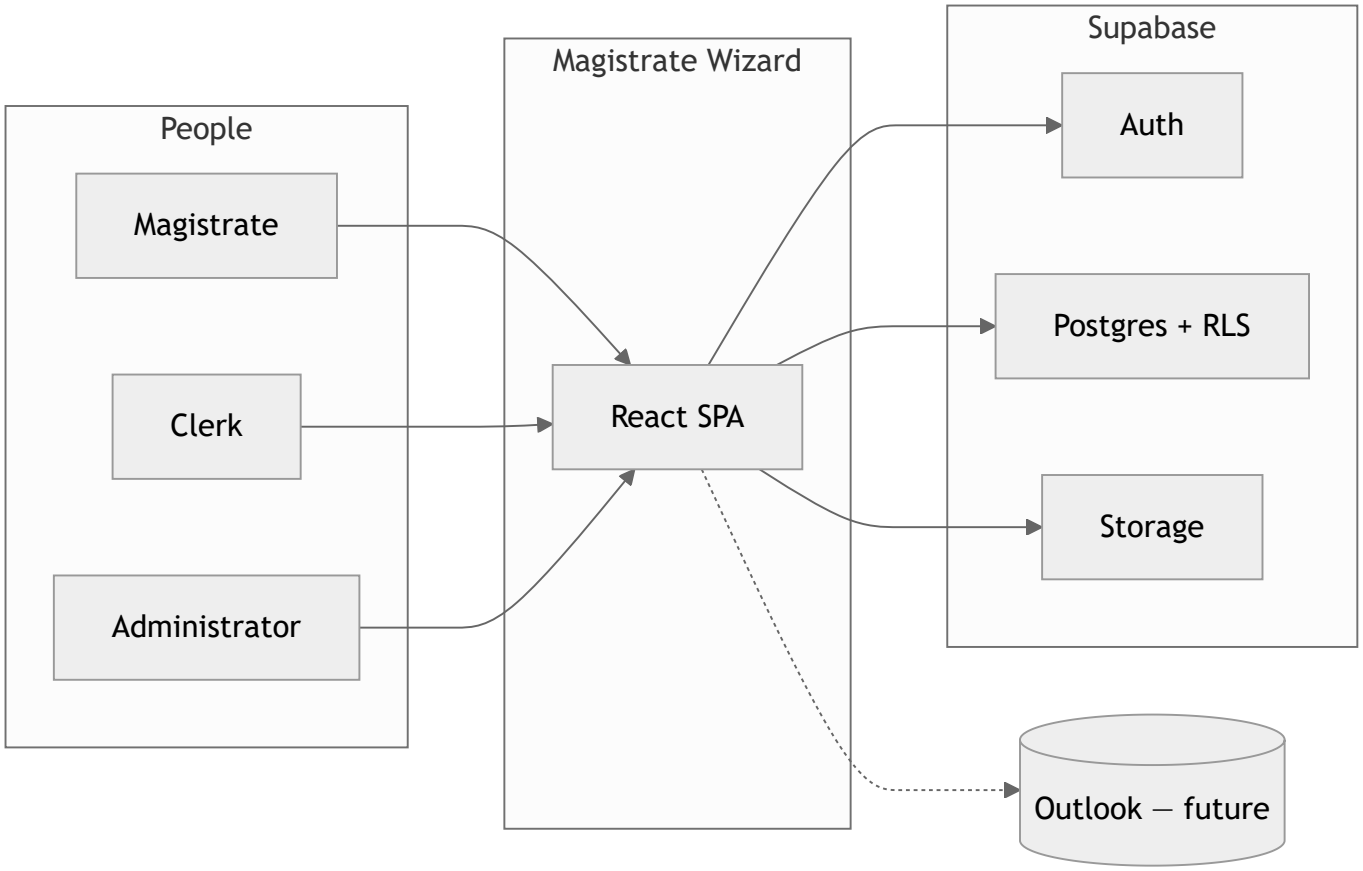
A sitting day versus an admin day

1. System context (C4 Level 1)

Magistrate Wizard is a single web application. Magistrates use it to run a Court docket, write judgments, and keep a private research library. Administrators assign magistrates to Courts and curate the shared legal library. Supabase is the only backend: Auth, Postgres with row-level security, and private file storage.

!System context

Outlook is a **future** calendar channel. It is not built. Docket Events remain the source of truth for when a matter is listed.



Plain reading

Think of Magistrate Wizard as a **courthouse filing cabinet with locks on every drawer**.

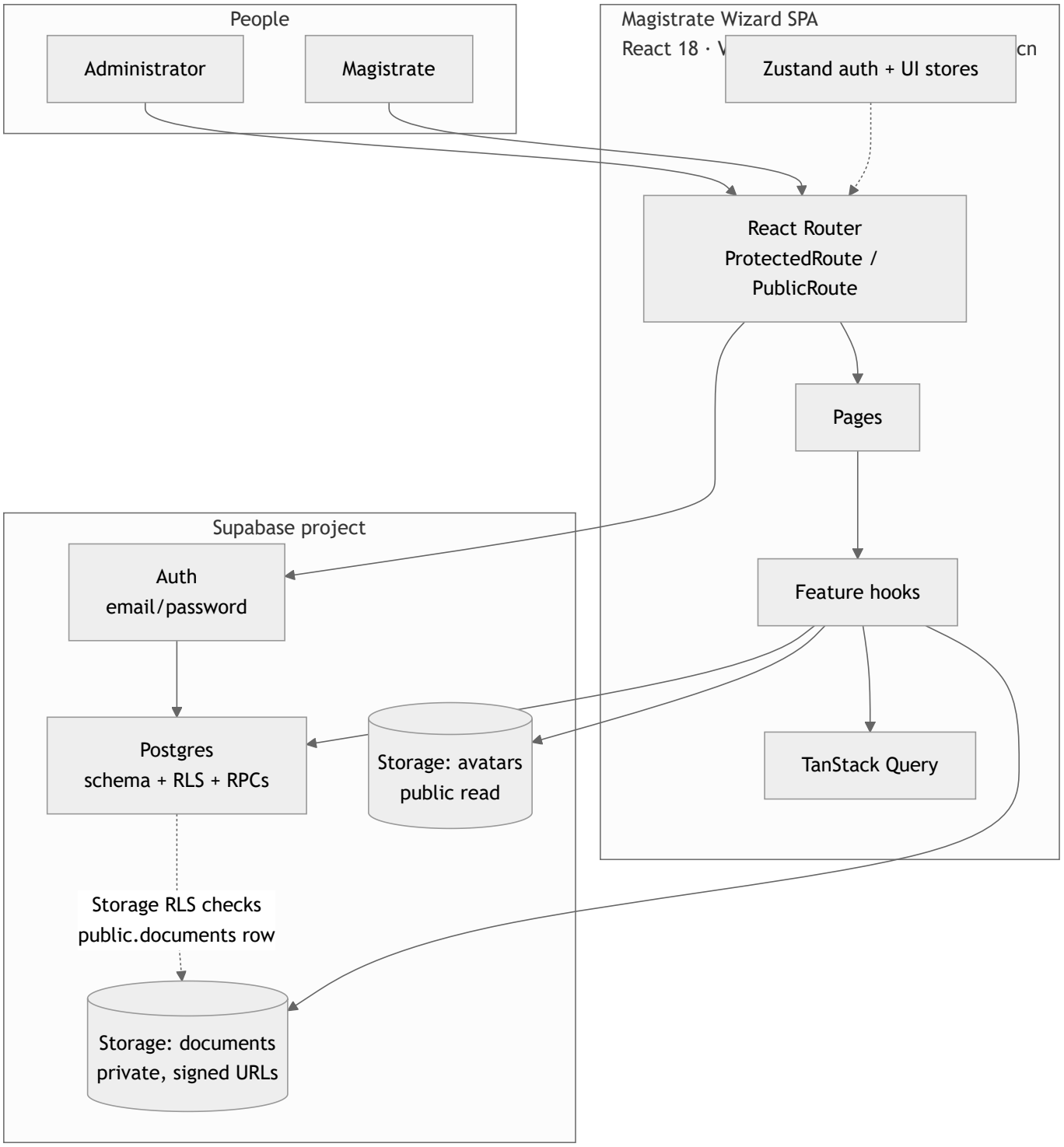
- The **Docket drawer** is the Court's. Anyone currently assigned to that Court can open it.
- The **Judgment drawer** is the magistrate's own. A colleague can read a judgment only if the author marks it discoverable.
- The **Library drawer** (statutes, canonical case law) is shared. Admins fill it; every magistrate may read published entries.
- Signing up does **not** hand you a Court. An administrator must assign you before you can create Docket Matters.

What is inside vs outside

Inside Magistrate Wizard	Outside / future
Docket, hearings, parties, shares	Outlook two-way sync
Judgments, Quick Codes, Bench Notes	AI extraction
Canonical + personal Case Law	Live URL crawlers
Legislation reader + ingestion review (including local scanned-PDF OCR)	Cloud document-AI OCR
Search (respects the same locks)	Public internet publishing

2. Containers (C4 Level 2)

One browser app, one Supabase project. There are **no Edge Functions** in production today. Search, access checks, and ingestion RPCs all run inside Postgres.



Container responsibilities

Container	Responsibility	Not responsible for
SPA	Screens, forms, signed-URL display, optimistic UI	Enforcing privacy. RLS is the real lock
Auth	Session, signup, password reset	Court assignment. Signup creates a profile with **zero** Courts
Postgres	Schema, RLS, triggers, search, ingestion RPCs	File bytes
`documents` bucket	PDFs, images, covers, identification photos	Deciding who may read. A matching `documents` row + parent RLS is required
`avatars` bucket	Profile pictures	Judicial content

Search as a container capability

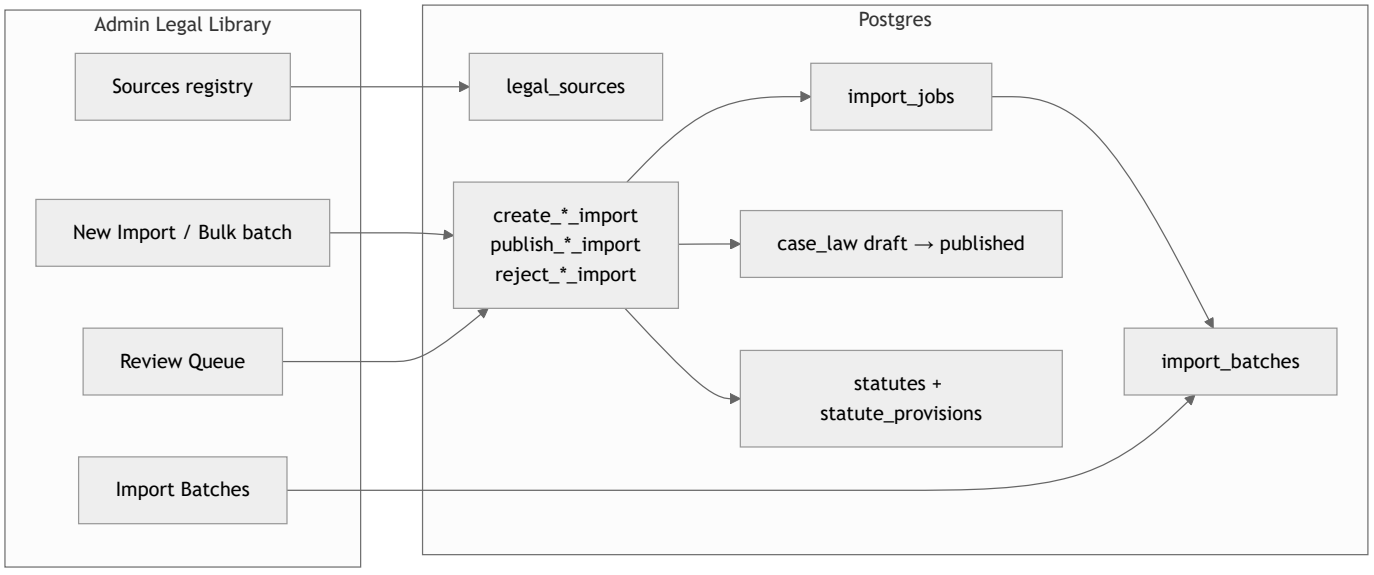
`global_search(query, limit)` is a `SECURITY INVOKER` SQL function. It unions:

`case` (legacy) · `bench_note` · `statute` · `case_law` · `docket_matter` · `judgment` · `quick_code`

Each branch only returns rows the caller could already `SELECT`. Search cannot leak a private Judgment or another Court's Docket. The Search page groups those seven types in JavaScript; it is not a SQL `GROUP BY`.

`search_statutes` (Legislation list) also matches **provision** body text. `global_search`'s statute branch does **not** — provision-only hits can appear on `/legislation` and miss on `/search`.

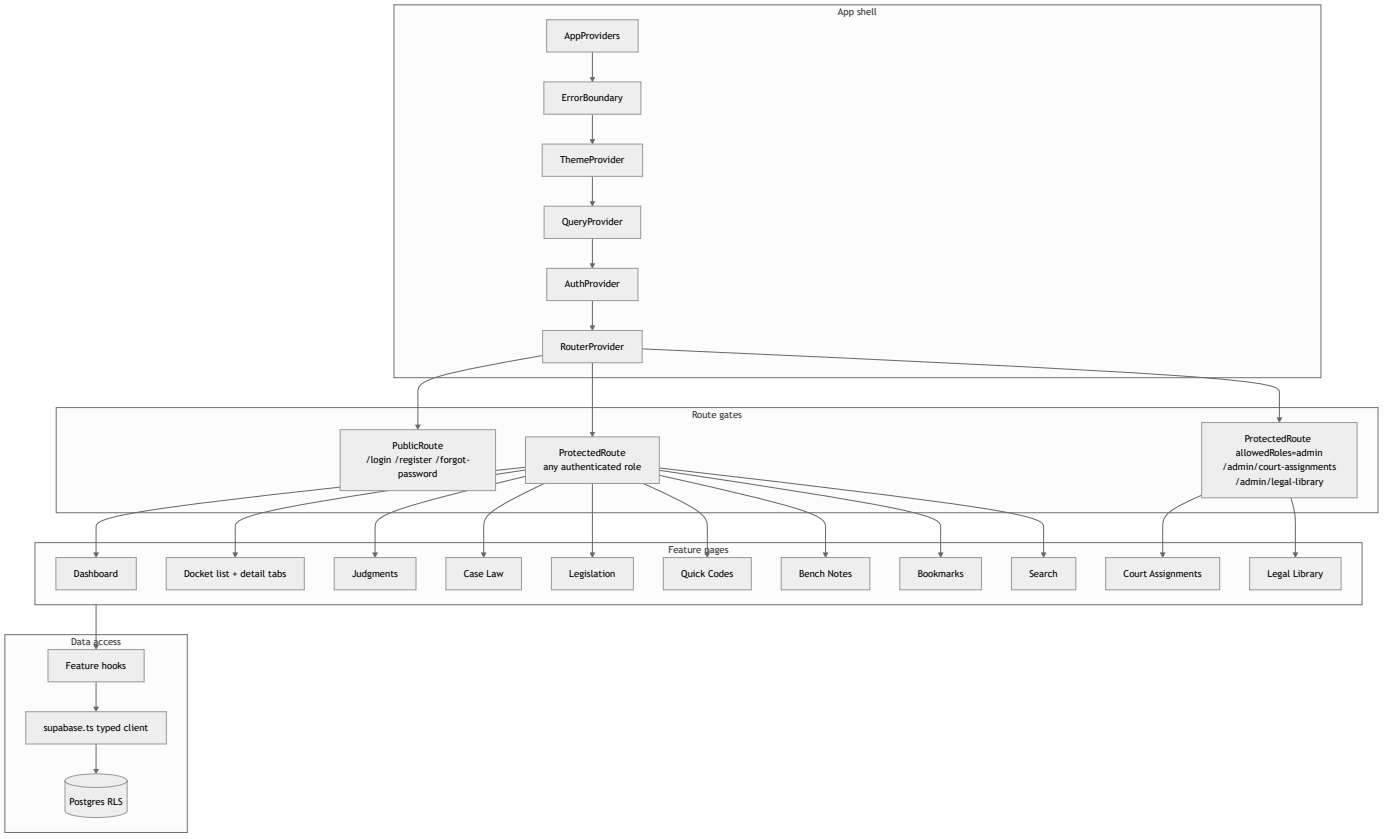
Ingestion machinery (admin only)



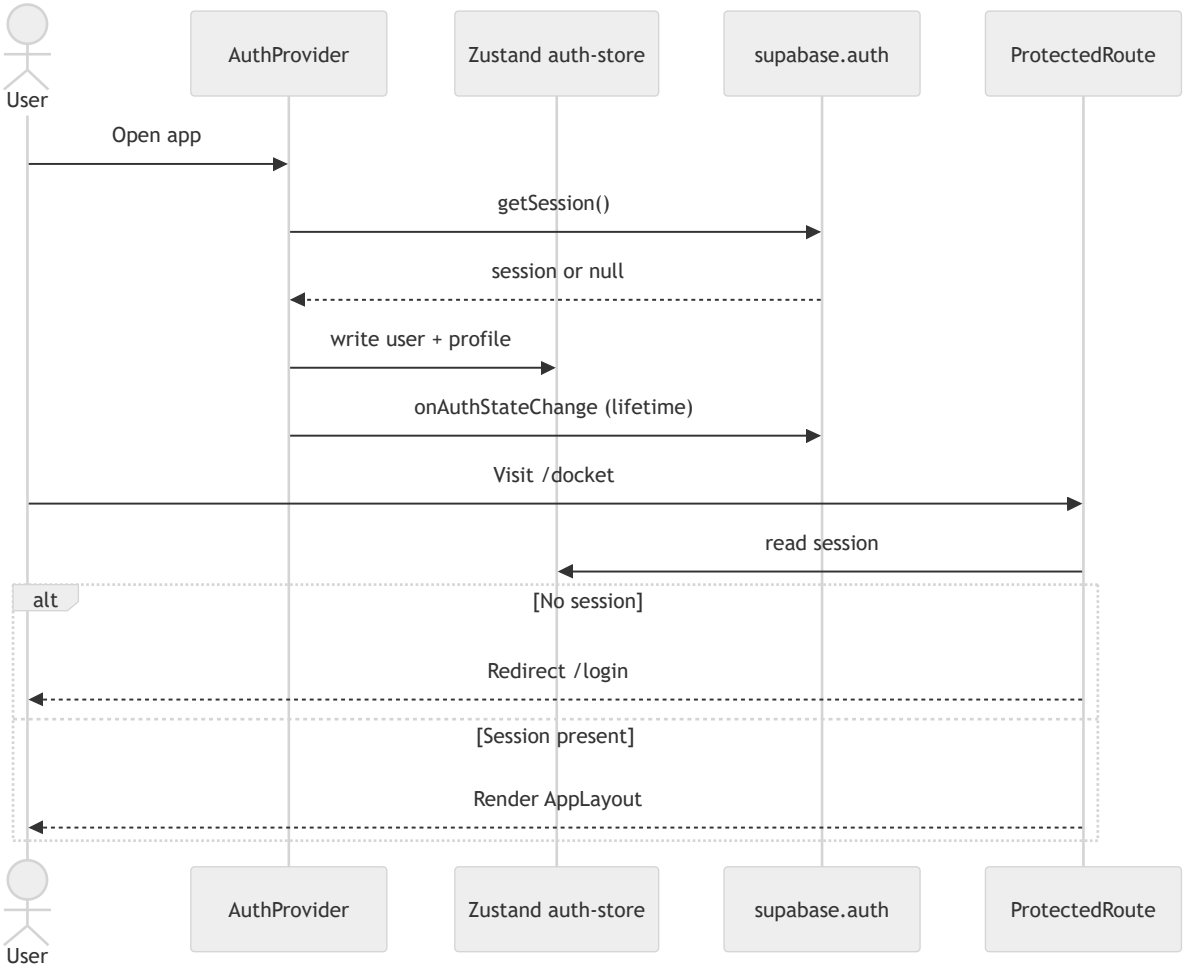
`legal_sources` is a **registry of intent**, not a running crawler. URL fetch and AI classification are not built. Scanned-PDF OCR **is** built in the browser (pdf.js rasterize + Tesseract.js, local only).

3. Application layers (C4 Level 3)

The SPA never invents access rules. Hooks call Supabase; Postgres RLS decides what comes back. Some editors remain clickable for a view-only share; the write still fails.



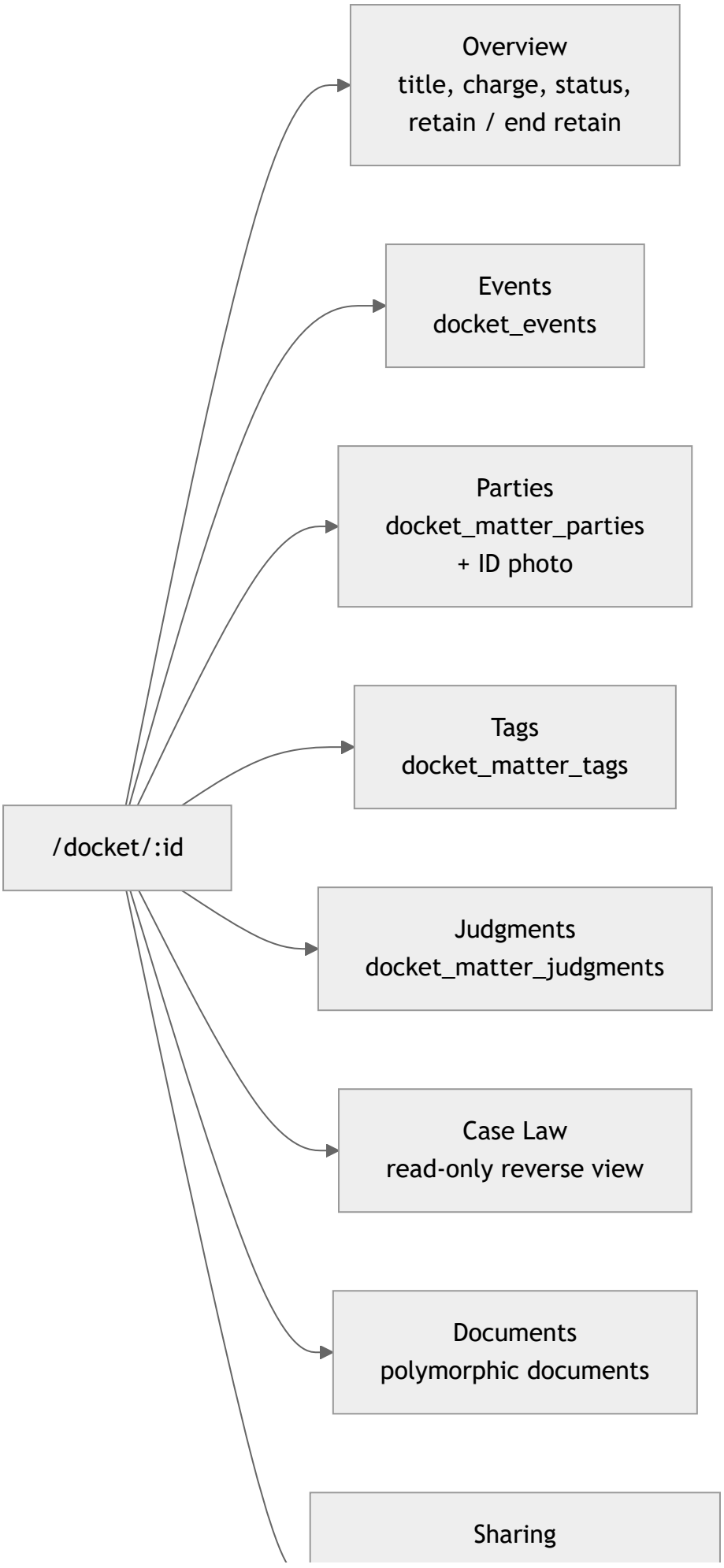
Auth flow

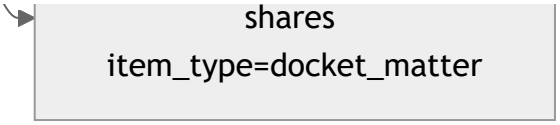


`useAuth()` is the only mutation surface for sign-in, sign-up, sign-out, and password reset. Pages do not talk to Auth directly.

Docket detail — real tabs

The matter page is the operational workspace. Tabs map 1:1 to child tables or association tables.


/docket/:id
Overview
title, charge, status,
retain / end retain
Events
docket_events
Parties
docket_matter_parties
+ ID photo
Tags
docket_matter_tags
Judgments
docket_matter_judgments
Case Law
read-only reverse view
Documents
polymorphic documents
Sharing



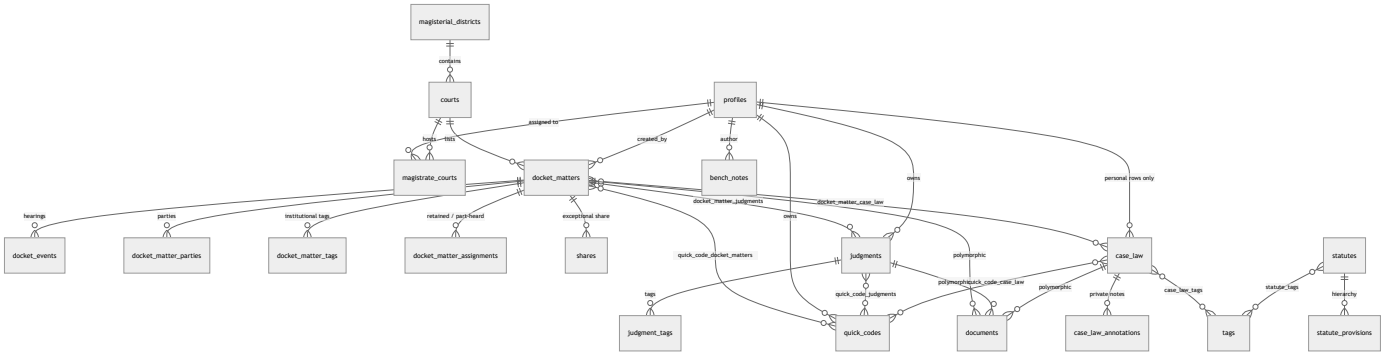
Provider / store facts

Layer	Module	Role
Composition	`src/providers/app-providers.tsx`	Order: Error → Theme → Query → Tooltip → Auth
Session	`src/store/auth-store.ts`	Mirrors Supabase user + profile
Chrome	`src/store/ui-store.ts`	Sidebar, mobile nav, command palette
Routes	`src/routes/router.tsx`	Admin routes are a separate `allowedRoles={["admin"]}` tree
Nav	`src/components/layout/nav-config.ts`	Court Assignments and Legal Library are admin-only items

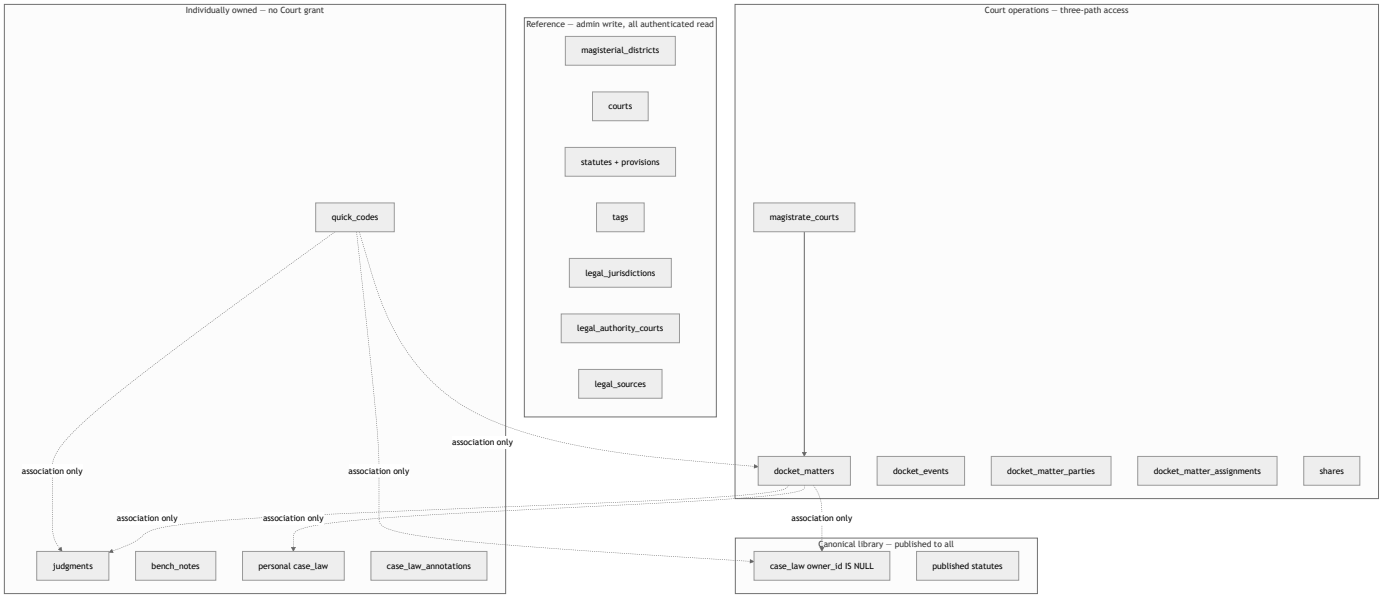
4. Complete ERD — overview

High-level map. Detail lives in the next three diagrams. Legacy `cases` / `case_parties` / `case_tags` exist in the database but are **not used** by new development.

!Entity map



Domain clusters



Cardinality cheat sheet

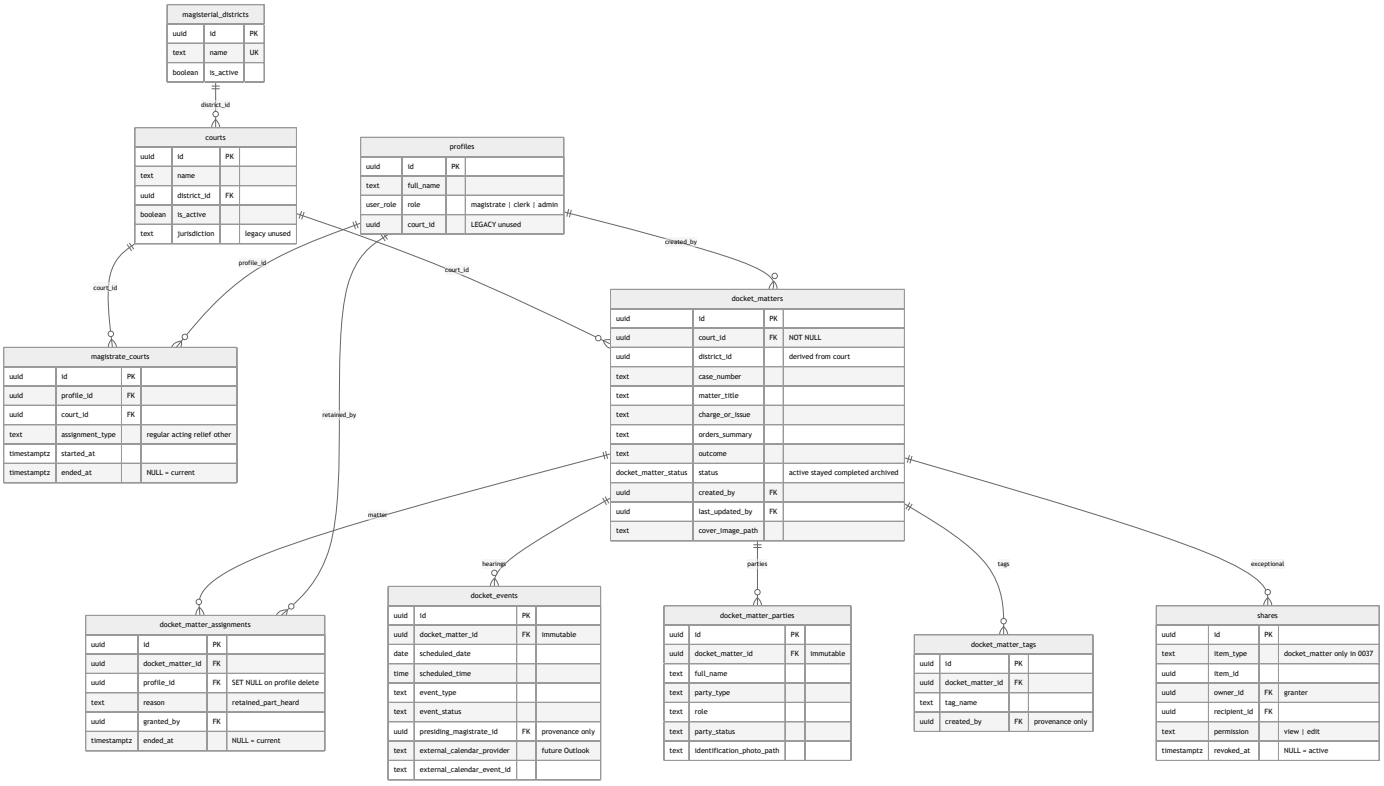
From	To	How
District	Courts	1 — many
Court	Magistrates	many — many via `magistrate_courts` (time-bounded)
Court	Docket Matters	1 — many
Matter	Events / Parties / Tags / Assignments	1 — many
Matter	Judgments	many — many, association only
Matter	Case Law	many — many, association only
Magistrate	Judgments / Quick Codes / Bench Notes	1 — many, owned
Case Law	Annotations	1 — many, always private to the annotator
Statute	Provisions	1 — many, self-parented tree

Deliberate non-relationships

- `judgments.court_name` is **free text**, not an FK to `courts`. A written judgment may concern a historical or external court.
- `legal_authority_courts` is **not** `courts`. One is the deciding court of a reported case; the other is a physical Guyana Magistrates' Court used for Docket authority.
- `docket_matter_tags` and `judgment_tags` are **not** joins to the global `tags` table. Global tags are readable by everyone; private tag text would leak.
- `profiles.court_id` is **legacy**. Live assignment is `magistrate_courts.ended_at IS NULL`.

5. ERD — Court structure and Docket family

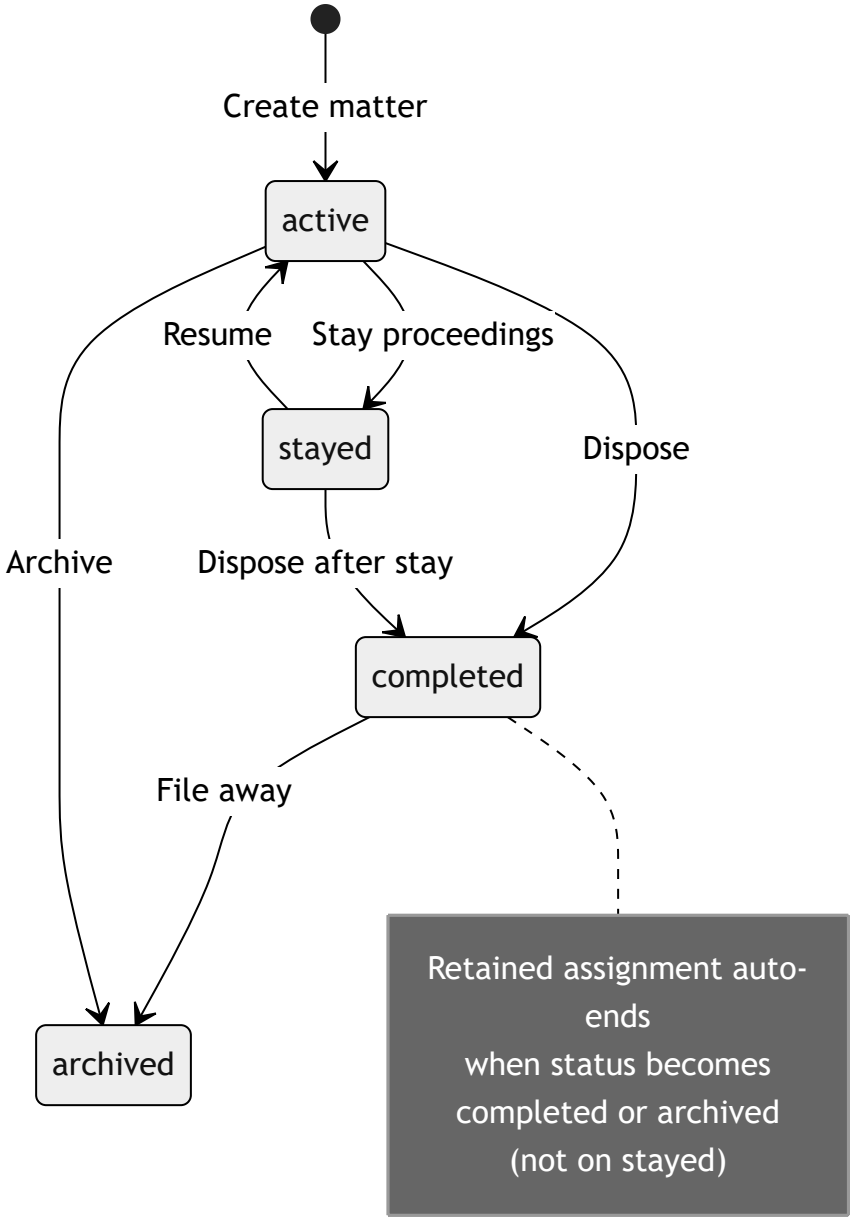
This is the operational heart of Magistrate Wizard. Access is **not** `owner_id`. Access is: current Court assignment **or** retained assignment **or** an active share.



Constraints that matter

Rule	Why
`UNIQUE (district_id, case_number)` on matters	Case numbers restart per Magisterial District. Not globally unique
`UNIQUE (profile_id, court_id) WHERE ended_at IS NULL`	One current assignment per magistrate/court pair. History kept
`UNIQUE (docket_matter_id, profile_id) WHERE ended_at IS NULL`	One live retained assignment per person per matter
At most one **active** share per recipient per matter	Soft-revoke (`revoked_at`), then create a new row to change permission
No `DELETE` policy on matters	Archive via `status`. Judicial history is not erased
`district_id` on a matter is trigger-derived	Client cannot pick a district that disagrees with the Court

Status of a matter



Identification imagery (0067)

Cover photos and party identification photos live in the **same private `documents` bucket**. Path columns on the matter/party are denormalized for list rendering. They do not grant access by themselves.

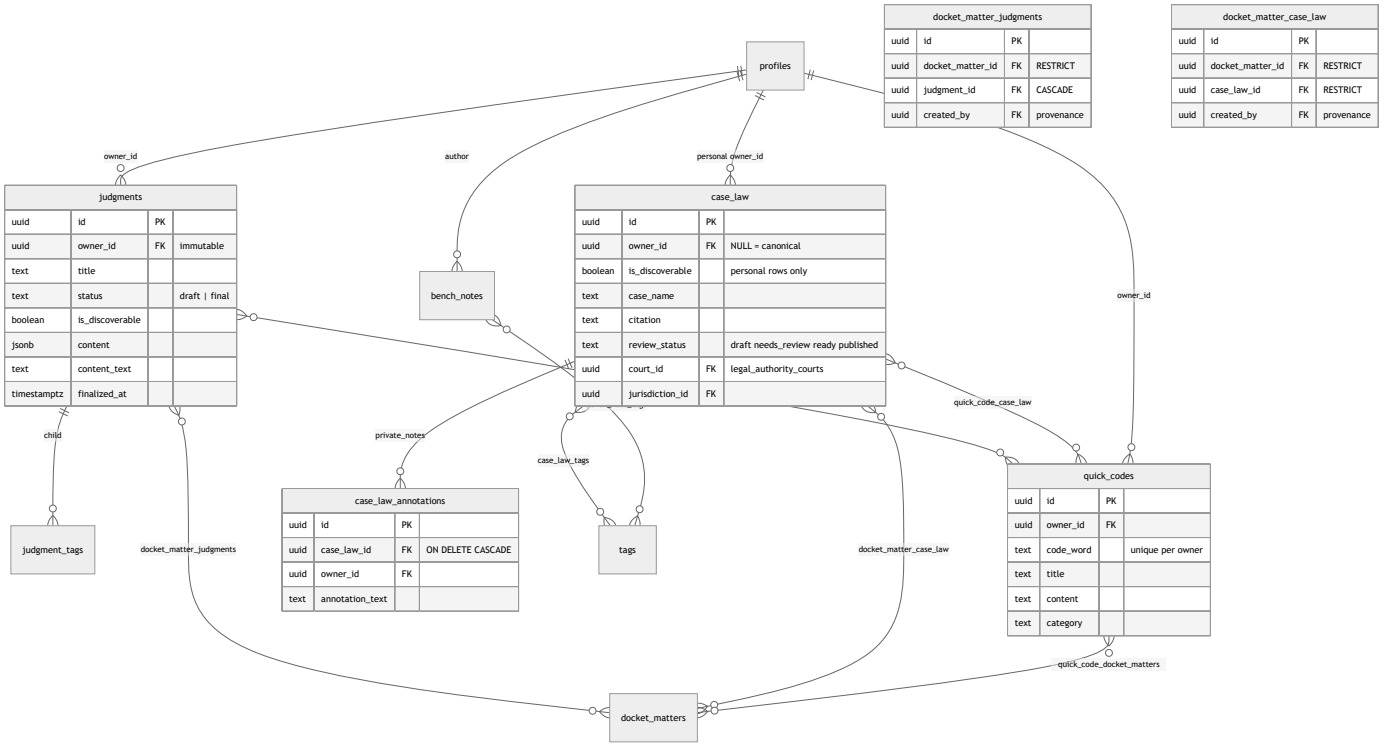
Column	Table	Purpose
`cover_image_path`	`docket_matters`	Browse tile / billboard
`identification_photo_path`	`docket_matter_parties`	Party photograph
`documents.purpose`	`attachment` / `cover` / `identification_photo`	Lets the Documents panel hide ID photos

Outlook boundary (not built)

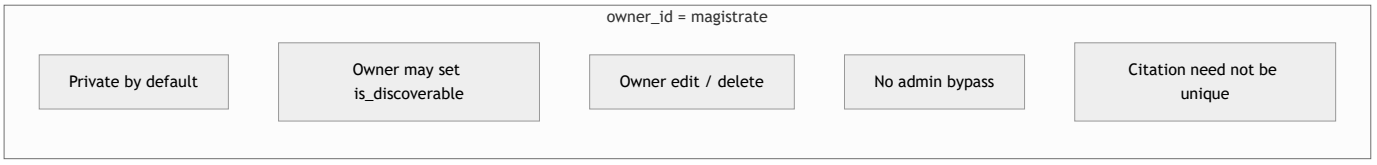
`docket_events` may later store `external_calendar_provider` + `external_calendar_event_id`. Outlook may supply **when and where**. It must never overwrite charge, parties, orders, or outcome.

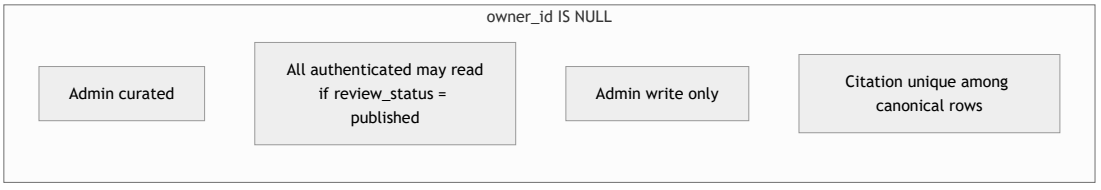
6. ERD — Judgments, Case Law, Quick Codes, notes

These records are **not** Court property. Linking them to a Docket Matter never opens the other side.



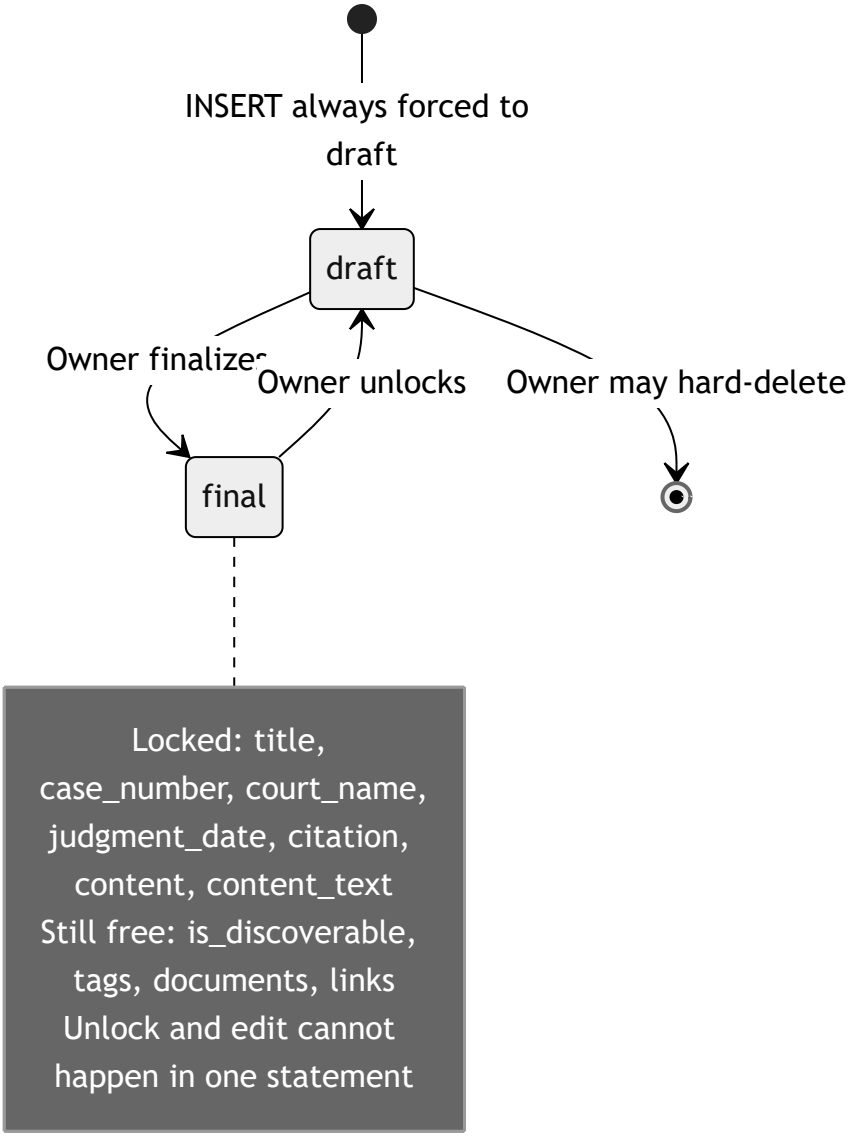
Two Case Law worlds in one table





Owning a Case Law record does **not** let you see another magistrate's annotations on it.

Judgment lifecycle

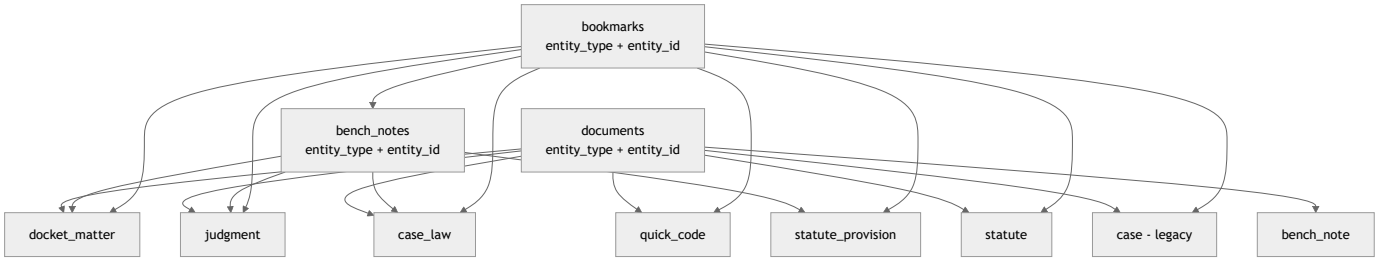


Association-table rules (BOTH sides, never OR)

Join table	SELECT	INSERT / DELETE	Parent delete
`docket_matter_judgments`	Docket access **and** Judgment read	Docket access **and** **Judgment ownership**	Matter RESTRICT, Judgment CASCADE
`docket_matter_case_law`	Docket **and** Case Law read	Docket **and Case Law read** (not ownership)	Both RESTRICT
`quick_code_docket_matters`	Quick Code owner **and** Docket	Same. Edit-share on Docket can mutate the link	Quick Code CASCADE, Matter RESTRICT
`quick_code_judgments`	Quick Code owner **and** Judgment read	Same	Both CASCADE
`quick_code_case_law`	Quick Code owner **and** Case Law read	Same	Quick Code CASCADE, Case Law RESTRICT

Discoverability of a Judgment can make a Quick Code↔Judgment link appear or disappear **without mutating the join row**.

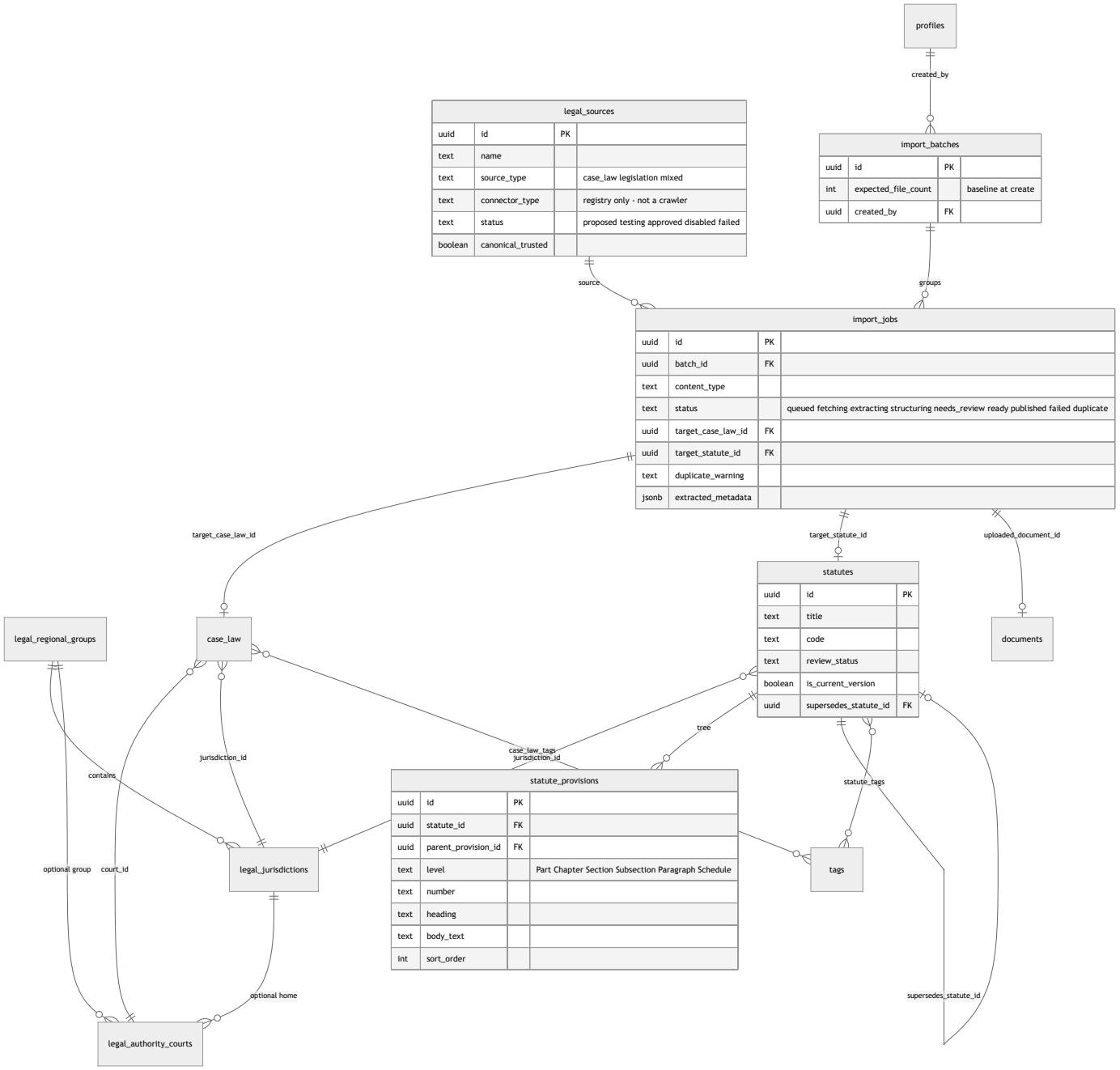
Polymorphic attachments



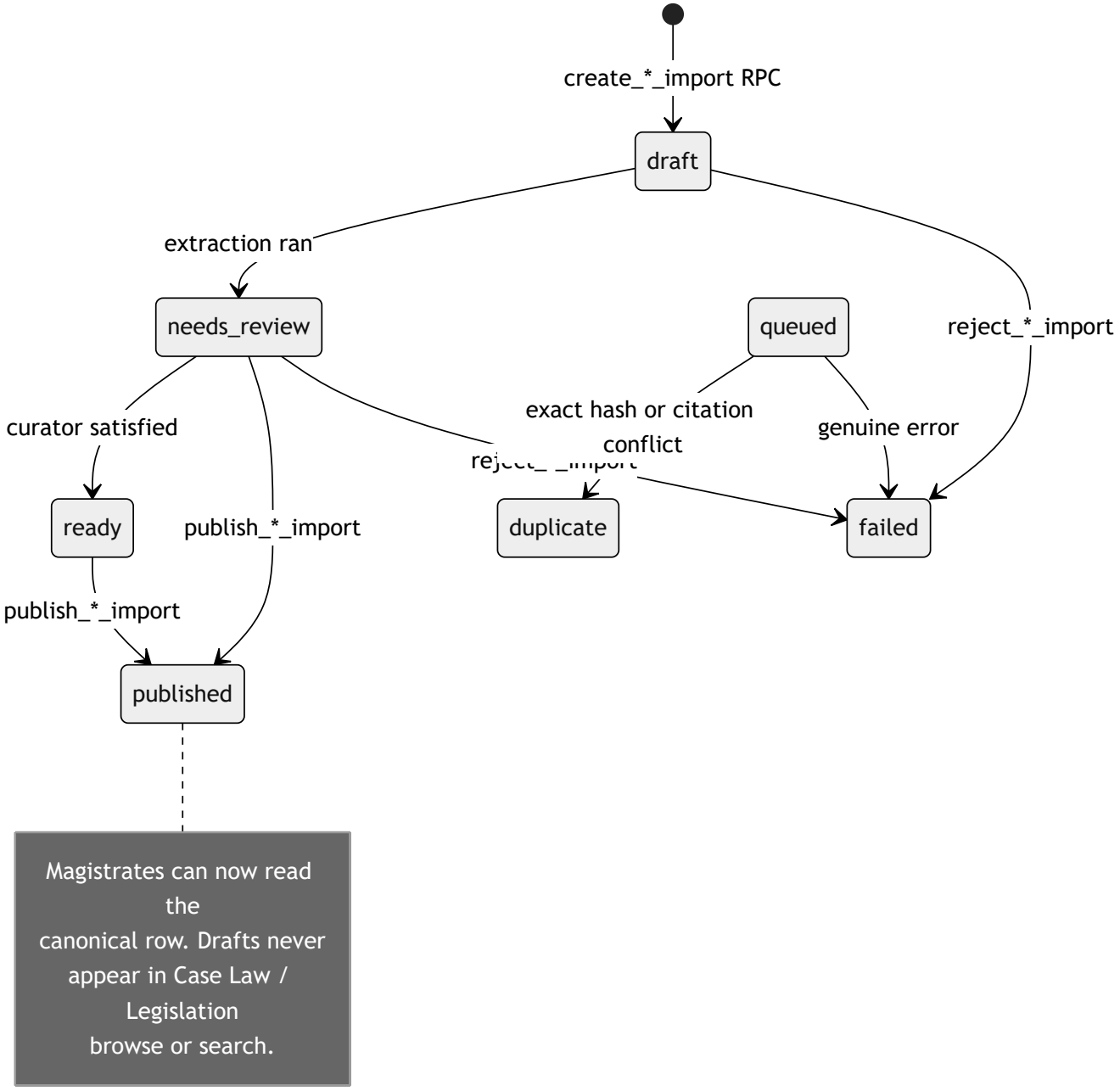
Bookmarks **cannot** target a `document` row. Attachments are bookmarked via their parent. Bench Notes are author-private even when the parent is widely readable.

7. ERD — Legal library ingestion and legislation

Canonical Case Law and Legislation are **administrative resources**. Drafts are invisible to ordinary magistrates until `review_status = published`.



Ingestion state machine



Draft-row-first (why)

A real `case_law` or `statutes` row is created immediately as `review_status = draft`. Publish is a status flip, not a copy. That lets the existing `documents` attachment model parent a file to a real id without a second storage row.

Publish RPCs reject placeholder metadata (for example `"Untitled (pending review)"`) so a record cannot go live empty.

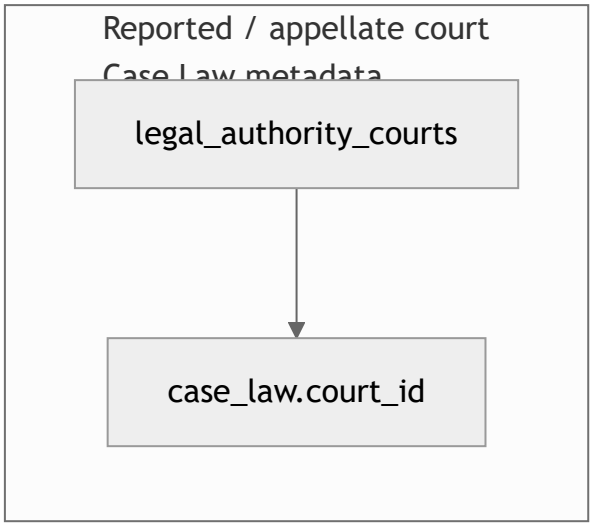
Duplicate is not failure

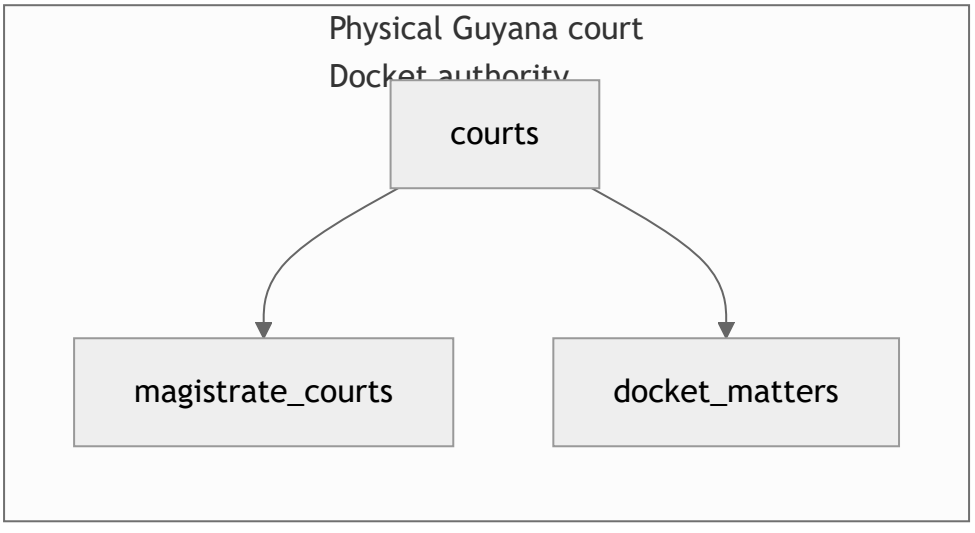
Bulk import records every file:

Outcome	`import_jobs.status`	Row created?
New draft	`needs_review` / `ready`	Yes
Byte-identical file	`duplicate`	No new authority; link to existing
Same citation, different file	`duplicate`	File attached to the **existing** Case Law as another source document
Validation reject / crash	`failed`	Bare job row so the batch still accounts for the file

`import_batches.expected_file_count` is stored at create time. A batch is fully accounted when `count(jobs) >= expected_file_count`. Older batches with a null count are labelled **legacy — incomplete history**.

Two different "courts"





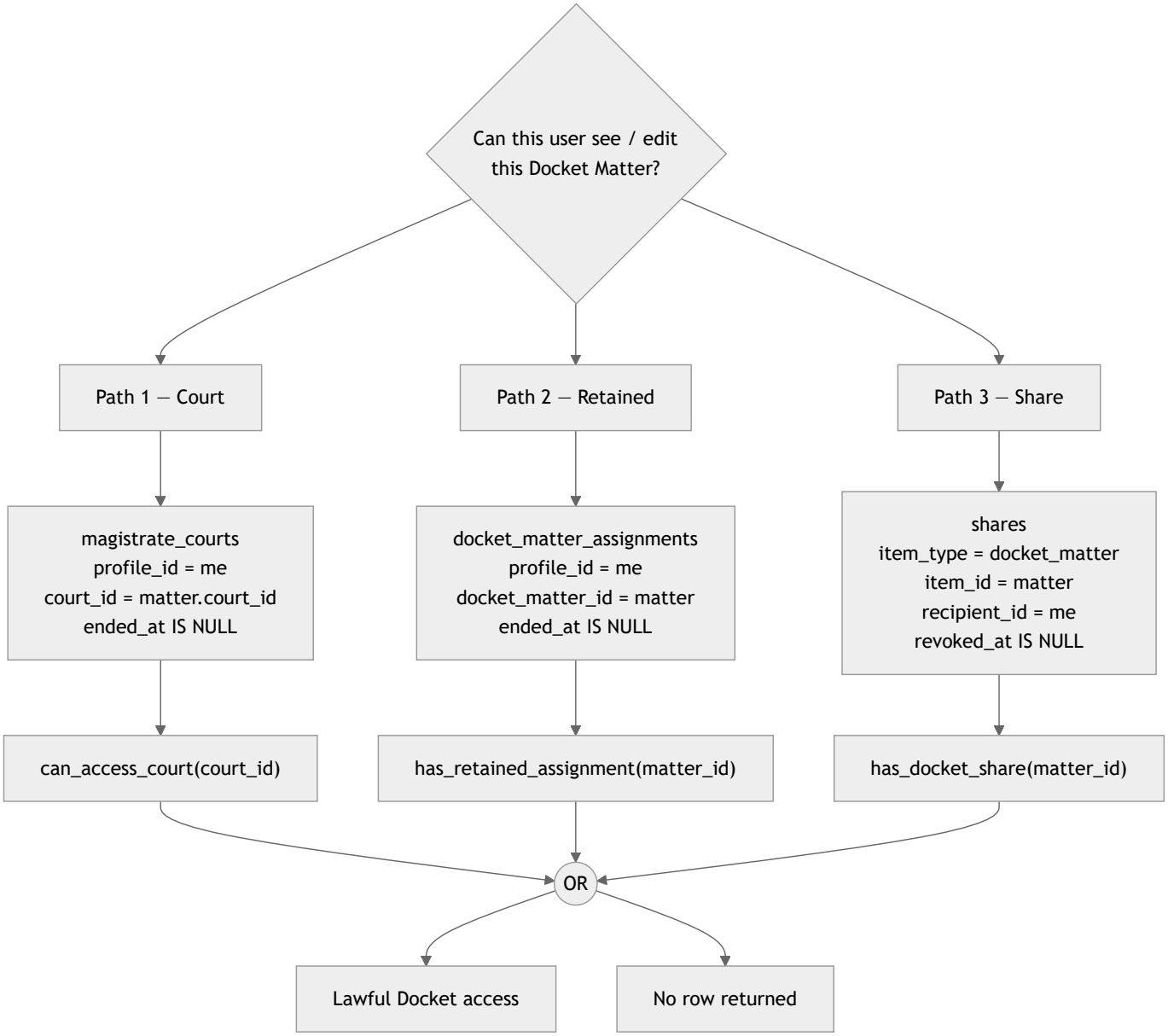
Never join these. "Vigilance Magistrate's Court" is a sitting venue. "Caribbean Court of Justice" is a deciding authority.

8. Access control and privacy

The security boundary is **Postgres RLS**, not the React UI. Some screens still show edit controls to a view-only recipient; the database refuses the write.

!Three paths onto a Docket Matter

Three paths onto a Docket Matter



`is_admin()` appears on **none** of these paths.

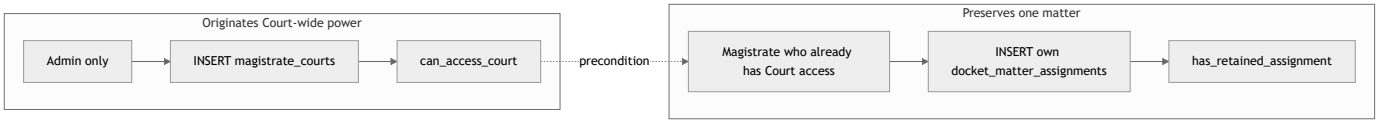
`has_docket_matter_authority()` is **not** a fourth read path. It is Court **or** retained only (never share). It is used to grant/revoke shares and to mutate Judgment / Case Law **links**. Share-only holders can see a matter; they cannot share it onward and cannot pin a Judgment or Case Law card onto it.

View vs edit on a share:

Permission	Parent matter	Events / parties / tags	Judgment / Case Law **links**	Quick Code ↔ Docket links
`view`	SELECT	SELECT	SELECT if BOTH sides already readable	SELECT only
`edit`	SELECT + UPDATE	Same mutation rights as a sitting magistrate on those children	SELECT only. INSERT/DELETE need Court or retained (`has_docket_matter_authority`), plus Judgment **ownership** to pin a Judgment	INSERT/DELETE allowed (the Quick Code must still be yours)

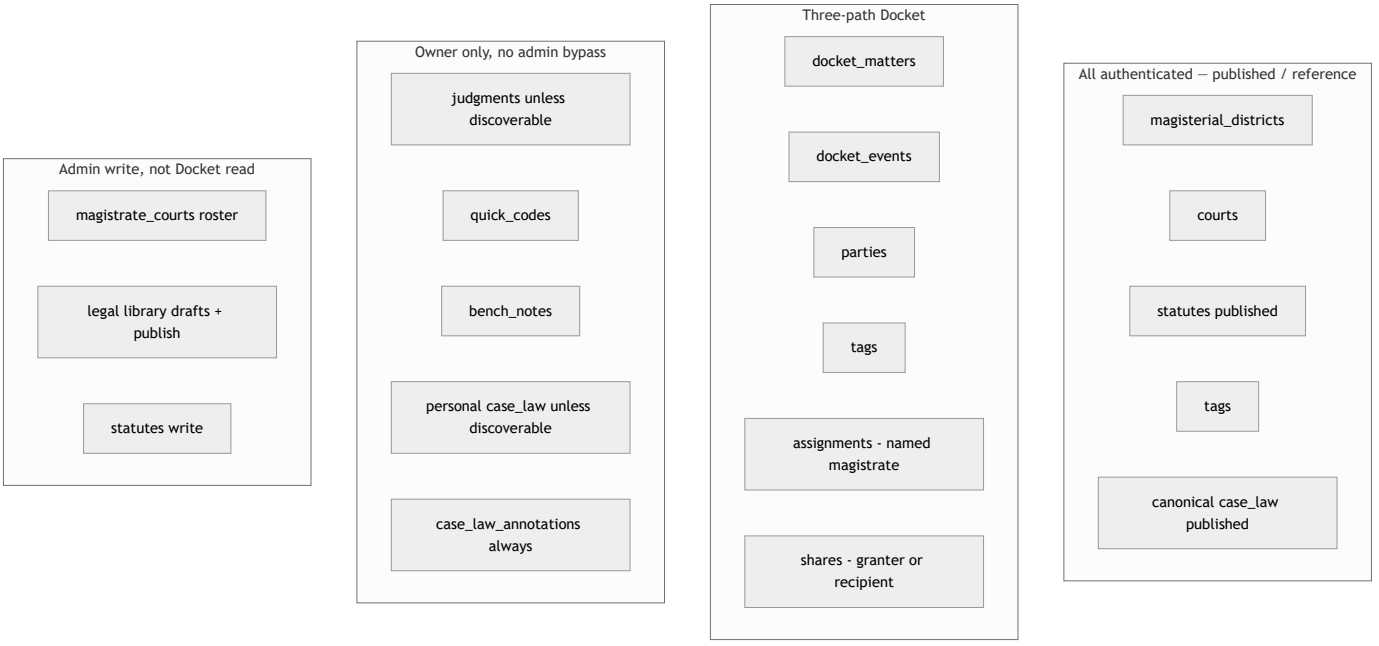
No resharing. Recipient may relinquish. Soft-revoke only. Live `shares.item_type` is **Docket only** (real FK, not a polymorphic uuid).

Originating authority vs preserving authority



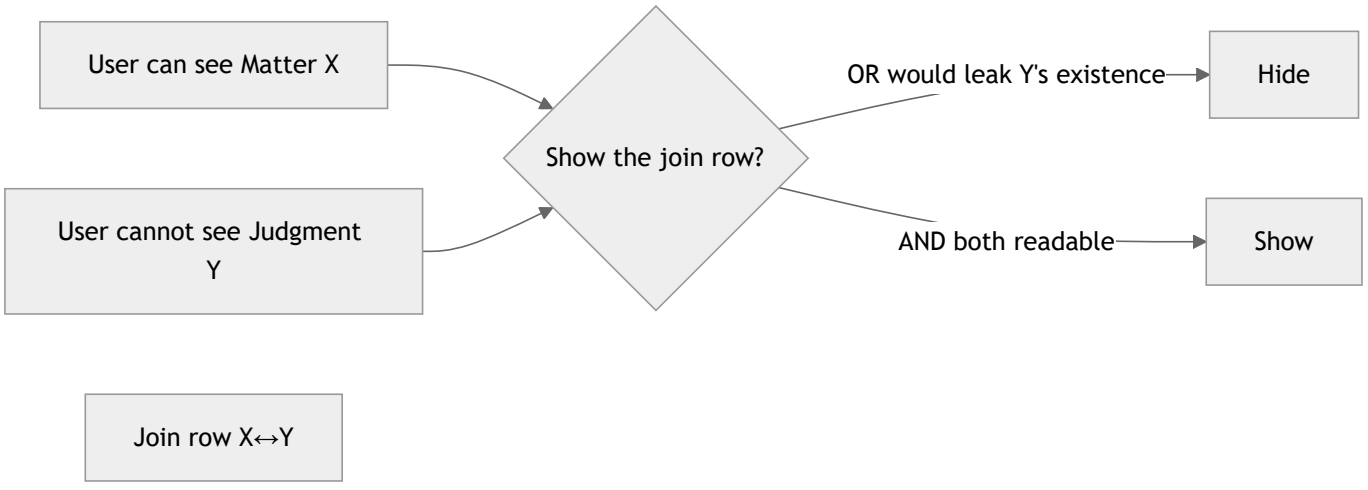
Self-assigning to a Court would let anyone grant themselves an entire docket. Self-retaining a matter you already sit is narrowing, not originating.

Visibility matrix



Entity	Default	Share?	Discoverable pool?	Admin bypass?
Docket Matter	Court-anchored	Yes, exceptional	**Never**	**No**
Retained assignment	Named magistrate	N/A (it is the grant)	No	**No**
Judgment	Private	Future (not in 0037)	Yes, read-only	**No**
Canonical Case Law	All authenticated (published)	N/A	N/A	Write yes
Personal Case Law	Private	Future	Yes, read-only	**No**
Annotation	Annotator only	Never	No	**No**
Quick Code	Owner only	No	**Never**	**No**
Bench Note	Author only	No	No	**No**
Documents	Follows parent	Follows parent	Follows parent	SELECT/INSERT follow parent. **DELETE** still allows `is_admin()` (metadata/blob only — not a Docket read bypass)
`magistrate_courts`	Self SELECT; Admin write	N/A	N/A	Yes (roster, not content)

Association side-channel rule



Always **AND**. Never **OR**.

Link mutation strengths (live SQL):

```
Docket ↔ Judgment:  Court-or-retained  AND  Judgment OWNERSHIP
Docket ↔ Case Law:  Court-or-retained  AND  Case Law READ
Quick Code ↔ Docket: Docket EDIT (edit-share OK)  AND  Quick Code OWNERSHIP
Quick Code ↔ Judgment / Case Law: Quick Code OWNERSHIP  AND  read access to the other side
```

The Quick Code association UI is **read-only today** — the tables and RLS exist; creating/removing those links is not wired in the SPA.

Roles in the product today

Role	UI	Database
`magistrate`	Full workspace	`profiles.role` is **not** consulted by Docket RLS. Authority is assignment / retain / share / ownership
`clerk`	Same nav as magistrate. Copy treats clerks as share recipients	**Enum leftover.** No Clerk policies. Identical RLS to a magistrate if they have the same assignment/share/ownership
`admin`	Extra: Court Assignments, Legal Library	Roster + canonical library write. **Zero** extra SELECT on Docket / private Judgment / personal Case Law / Quick Codes / Bench Notes

An administrator who also sits a Court needs a real `magistrate_courts` row, created the same way as anyone else's.

Profile privacy

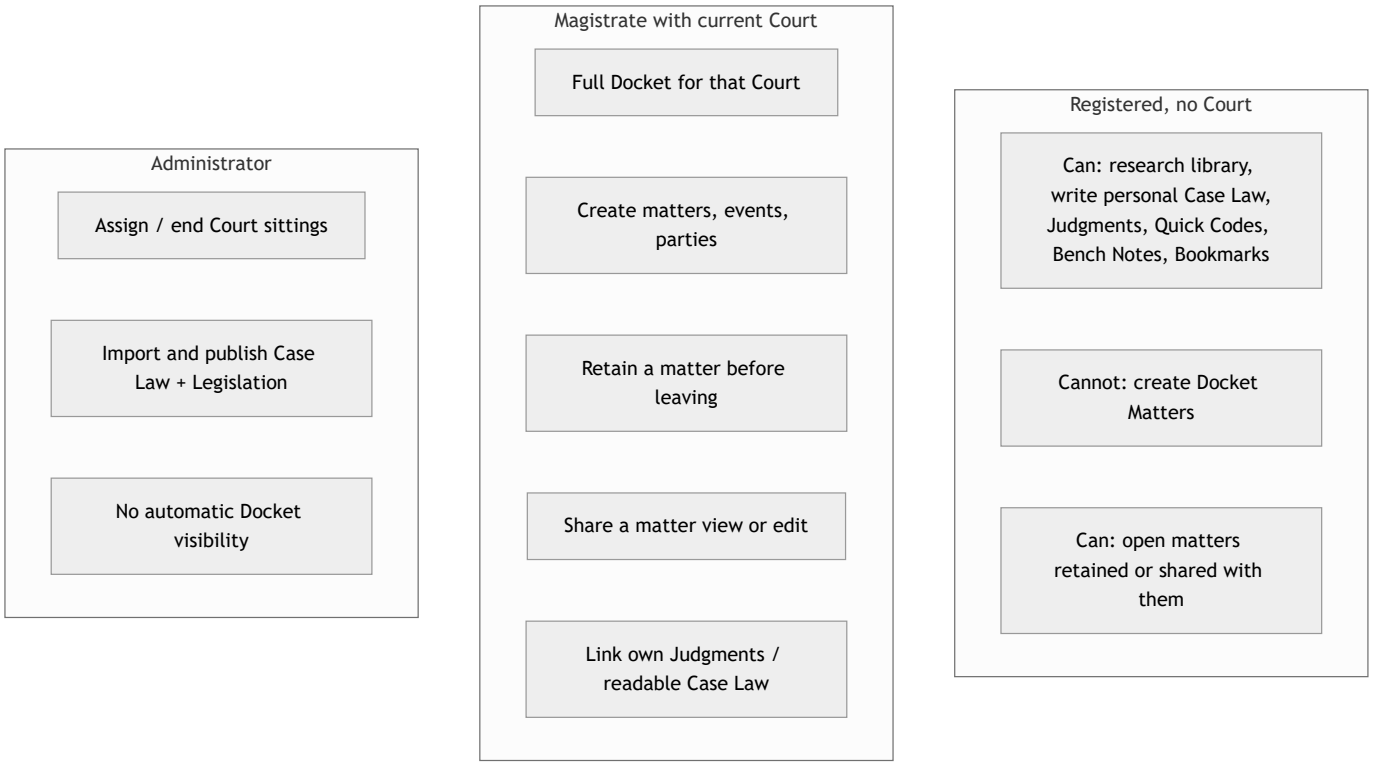
`profiles` SELECT is self-only. Display names on retained assignments and shares come from two narrow `SECURITY DEFINER` functions (`0043`), gated by context, never by opening the whole profiles table.

9. User journeys

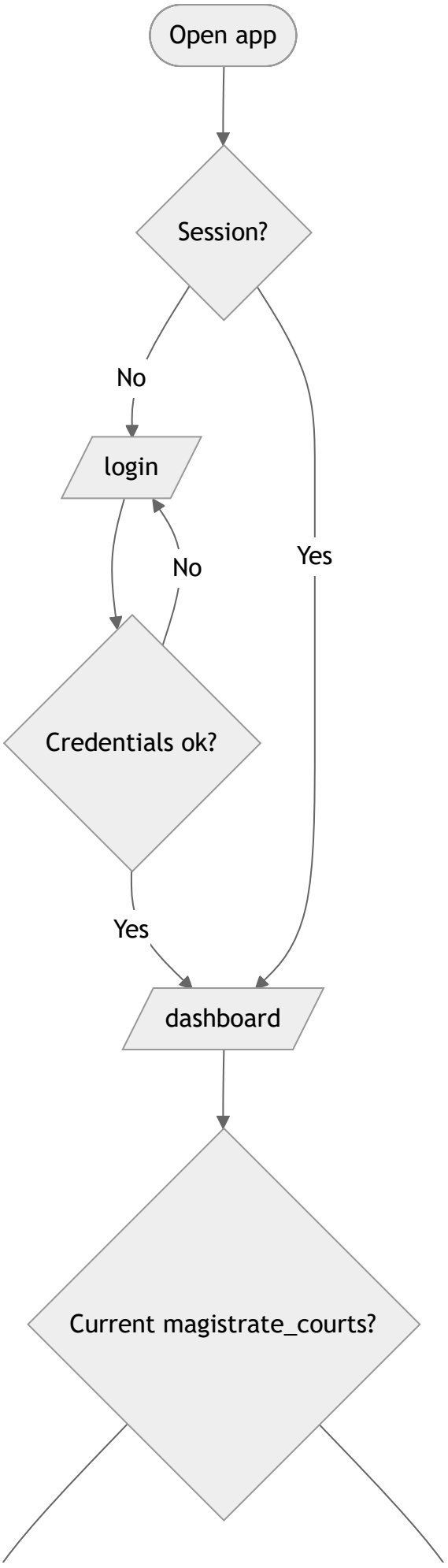
Actors, screens, and blocked paths as implemented in the SPA.

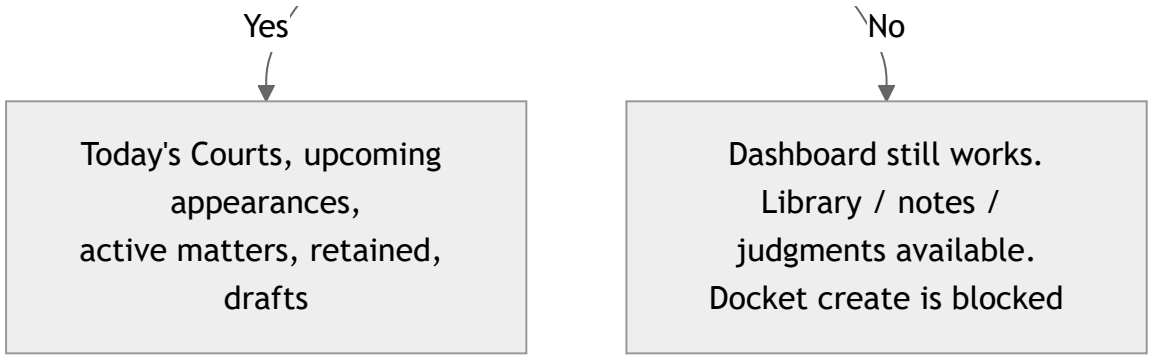
!A sitting day versus an admin day

Personas



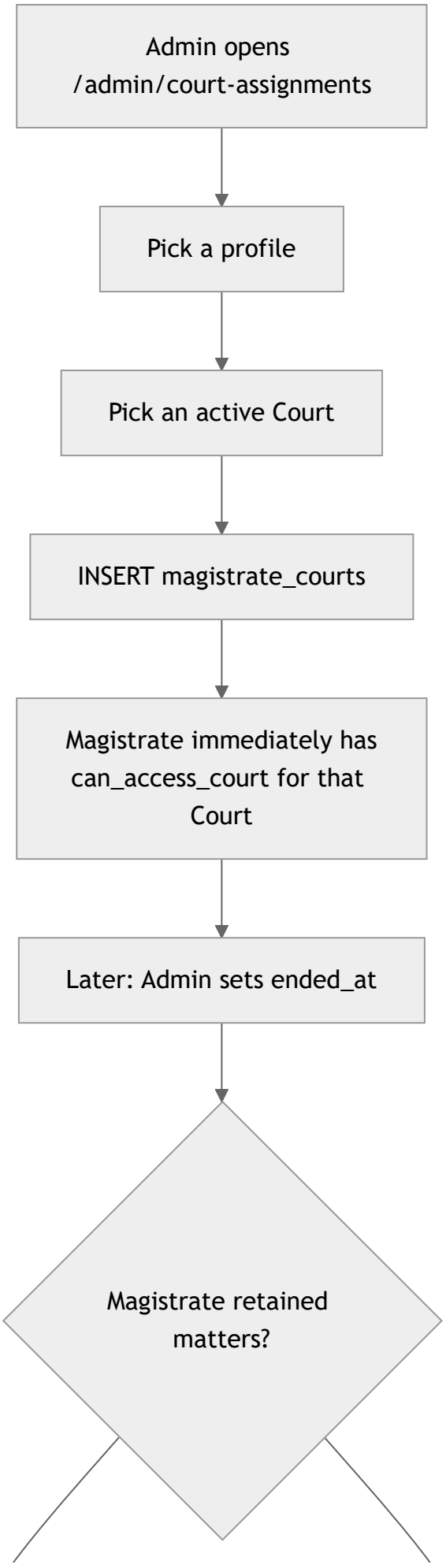
Journey A — First login


Open app
Session?
No
login
No
Yes
Credentials ok?
Yes
dashboard
Current magistrate_courts?



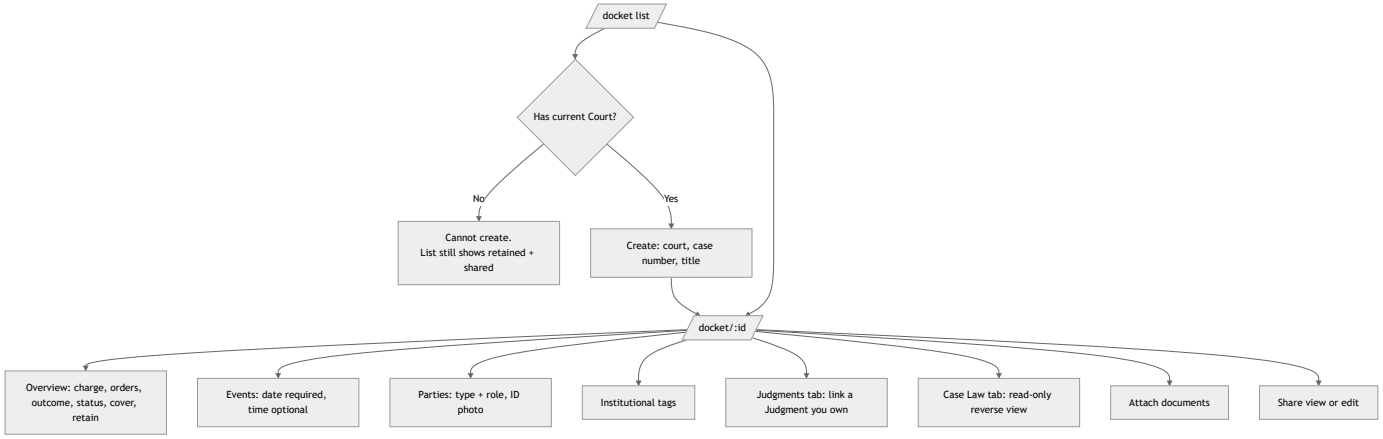
Register (`/register`) creates `auth.users` + `profiles` . It does **not** assign a Court.

Journey B — Sit a Court (admin)

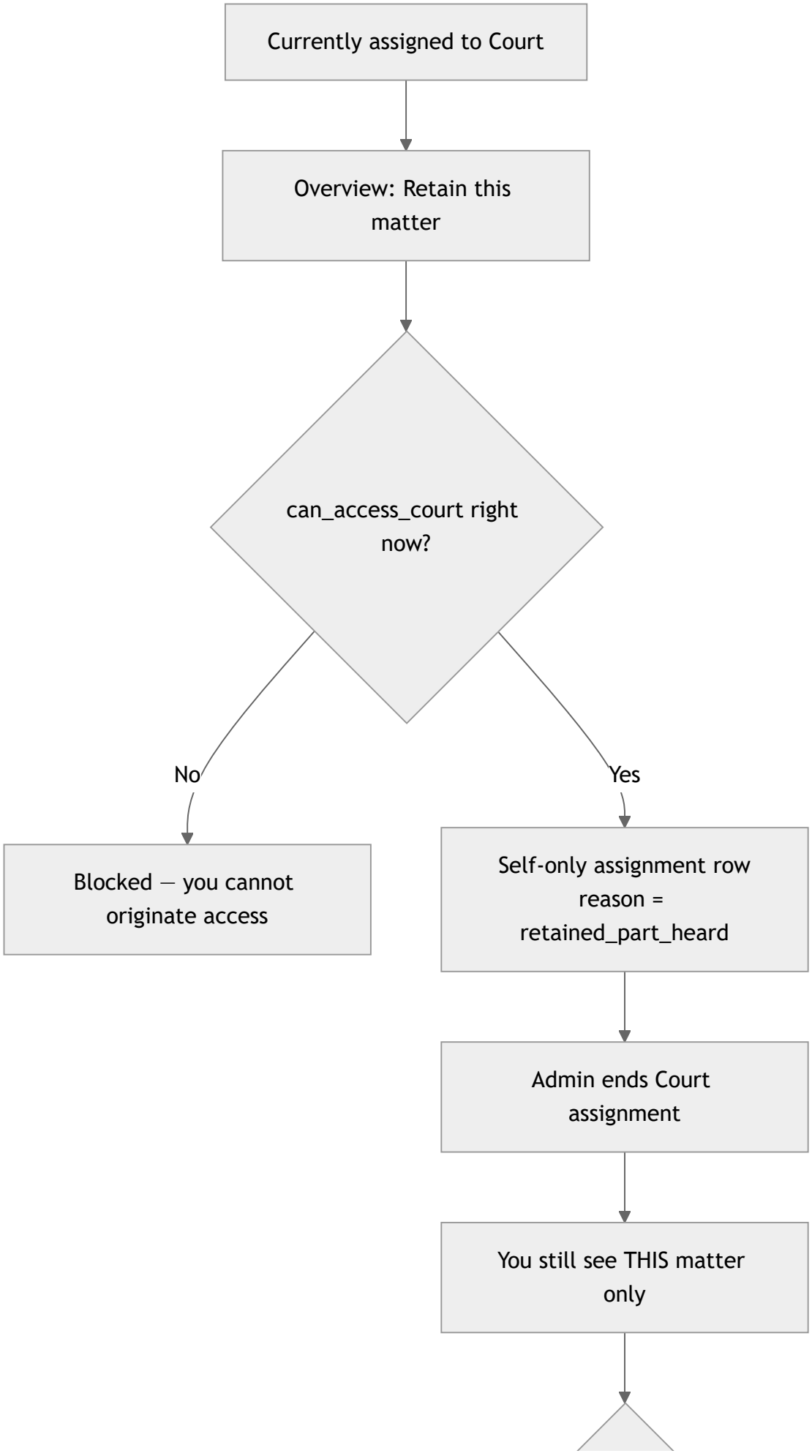

Admin opens /admin/court-assignments
Pick a profile
Pick an active Court
INSERT magistrate_courts
Magistrate immediately has can_access_court for that Court
Later: Admin sets ended_at
Magistrate retained matters?

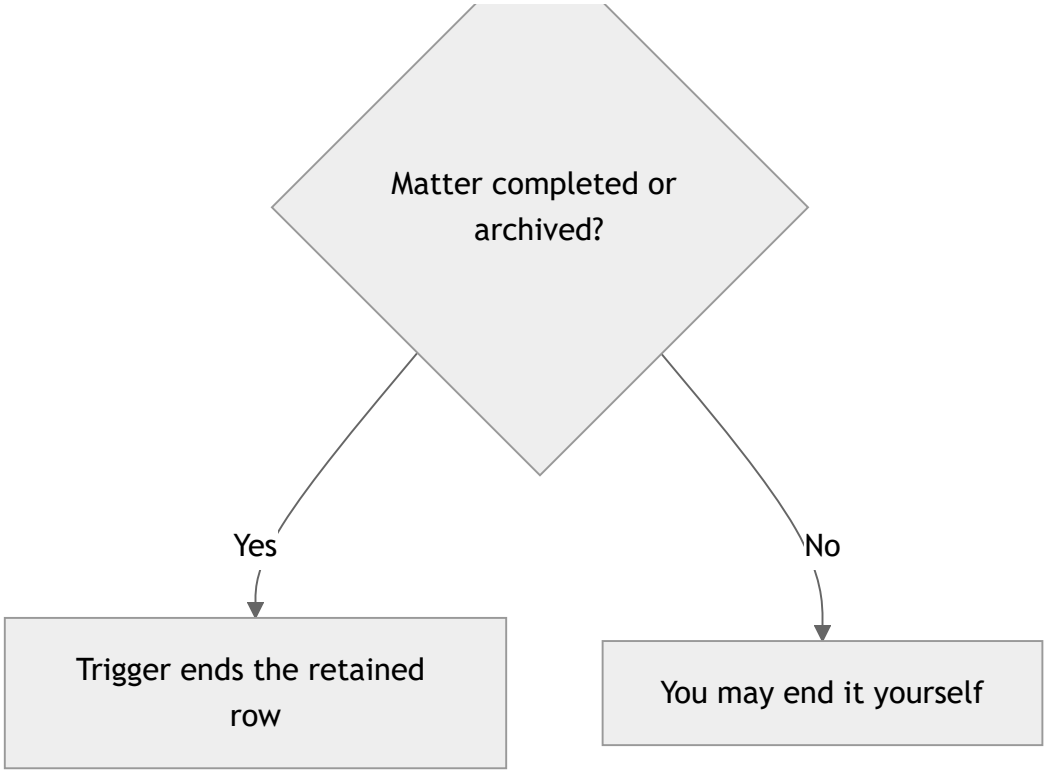
Magistrates cannot insert or end their own Court assignment in V1.

Journey C — Work a matter (magistrate)



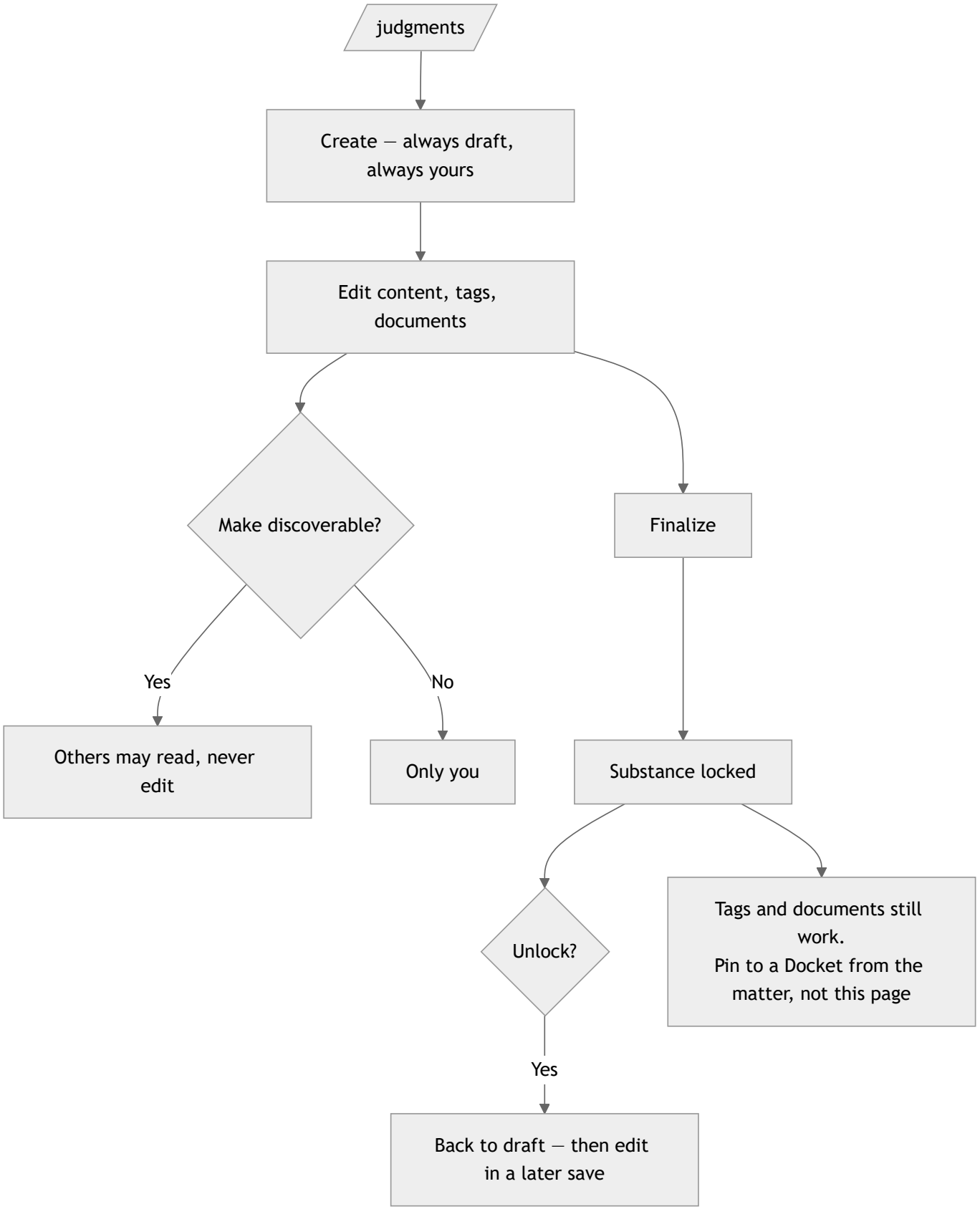
Journey D — Leave a Court but keep a part-heard matter


Currently assigned to Court
Overview: Retain this matter
can_access_court right now?
No
Yes
Blocked — you cannot originate access
Self-only assignment row reason = retained_part_heard
Admin ends Court assignment
You still see THIS matter only



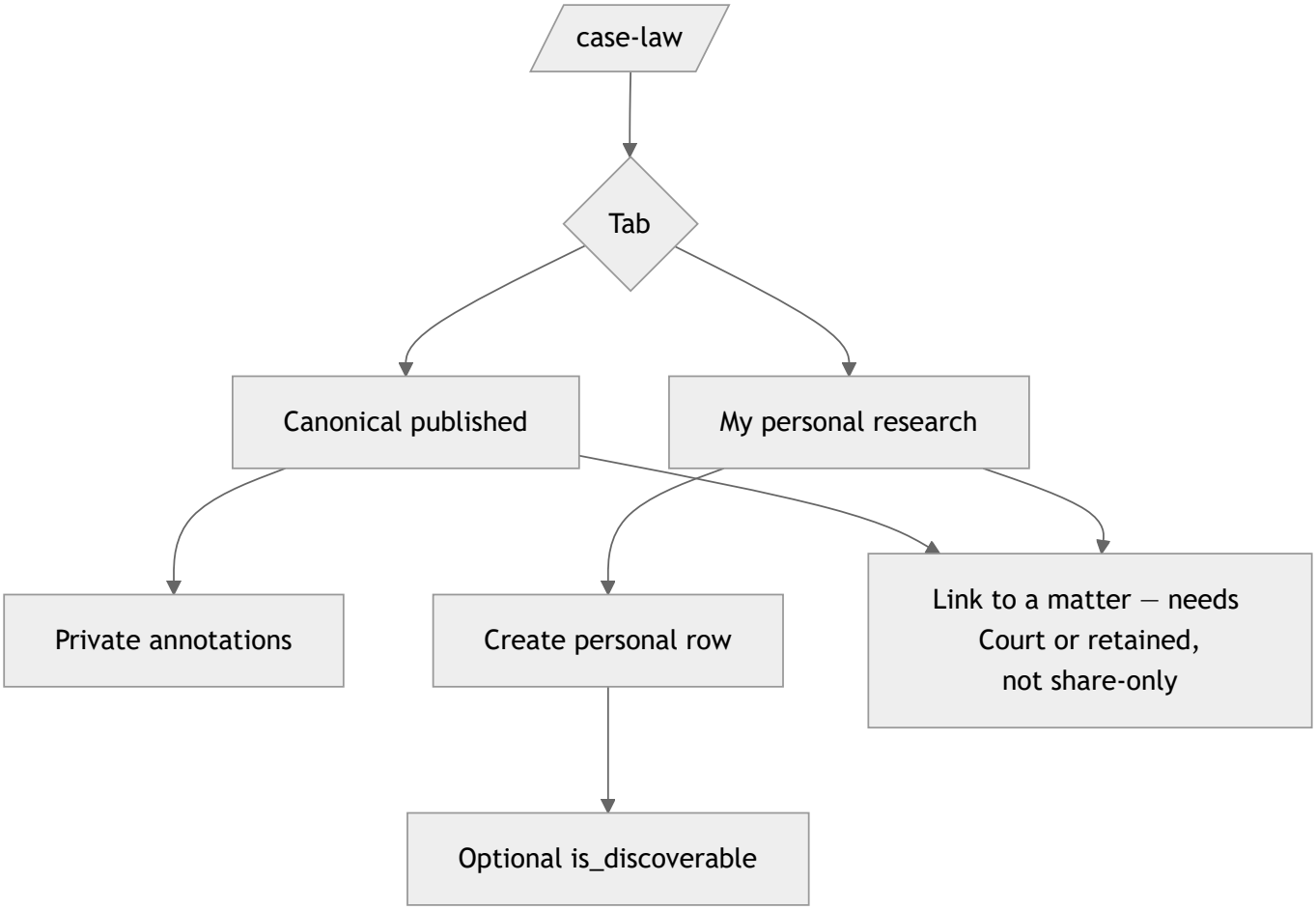
A successor assigned to the Court also sees the matter. Retained access is **not exclusive**.

Journey E — Write and finalize a Judgment



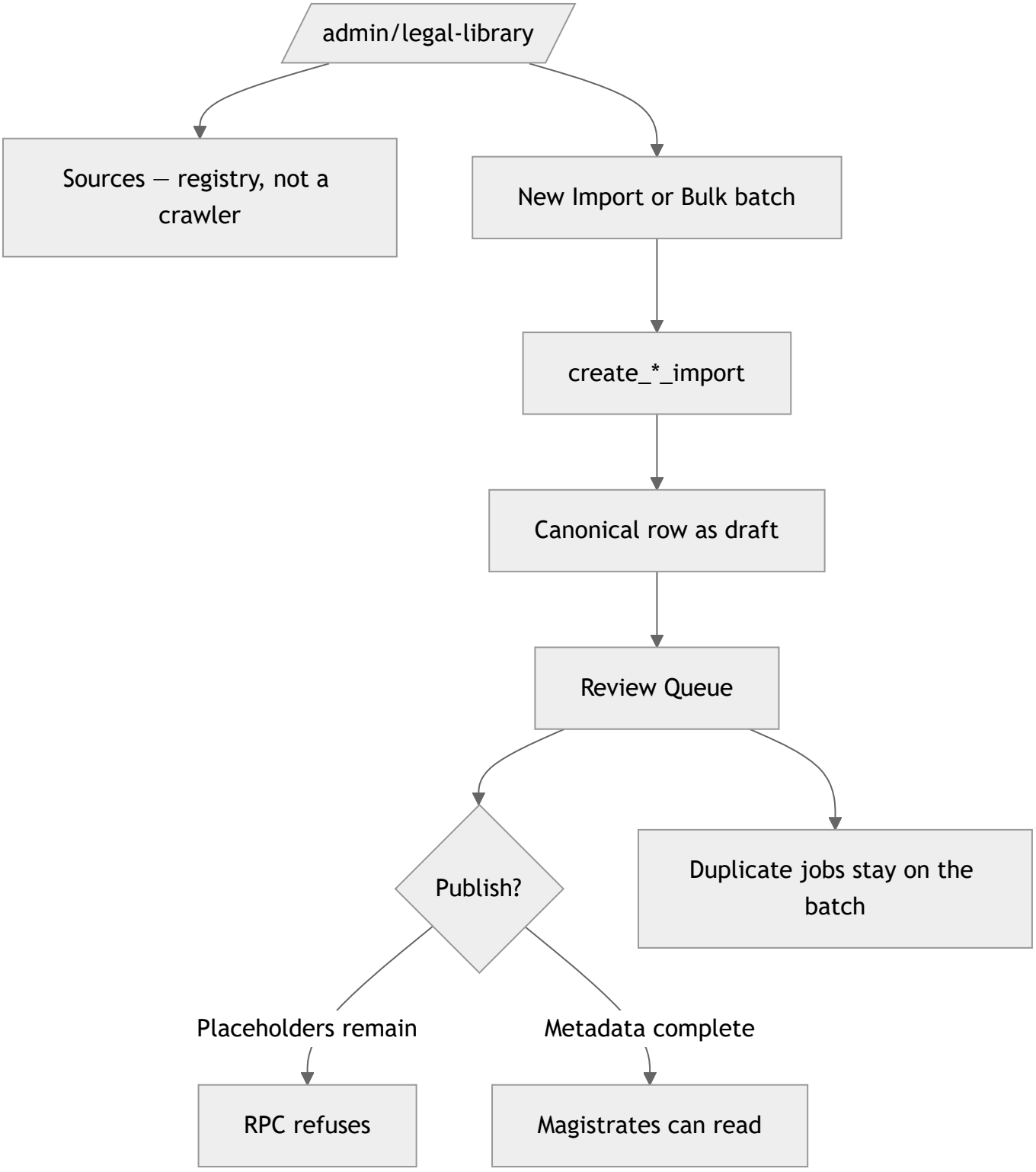
Leaving a Court does **not** transfer Judgments.

Journey F — Research Case Law



Unpublished library drafts never appear here. Admins review them at `/admin/legal-library`.

Journey G — Admin legal library



Journey H — Search and bookmarks

Global search (`/search`) groups **seven** SQL branches in the browser by `entity_type` . Every hit is something the caller could already open. Legacy `case` rows can appear with **no** detail route.

The Legislation **list** search matches provision body text. **Global** search's statute branch still searches only the Act-level `search_vector` — a phrase that lives only inside a section may miss in `/search`

and hit on `/legislation`.

Bookmarkable: Docket Matter, Judgment, Case Law, Quick Code, Bench Note, Statute, Statute Provision, leftover Case. **Not** a raw document file.

Honest gaps in the live UI

You might expect	What the SPA actually does
View-share disables editors	Controls stay enabled; RLS rejects the write with a toast
Link Case Law from the matter tab	Matter tab is read-only. Link from **Case Law** detail
Link a Judgment from the Judgment page	Links panel is read-only. Link from the **matter** Judgments tab
Type a Quick Code to expand in an editor	**Copy** copies `content` to the clipboard
Create Quick Code ↔ matter/judgment/case-law links	Associations dialog is **read-only**
New Bench Note on a docket / judgment / case-law page	Create from Bench Notes list, or from a legislation section
Password-reset email sets a new password in-app	Link lands on `/login`. No set-password page. Profile / Settings are disabled
Clerk-only screens	None. Clerk collapses onto the magistrate RLS envelope

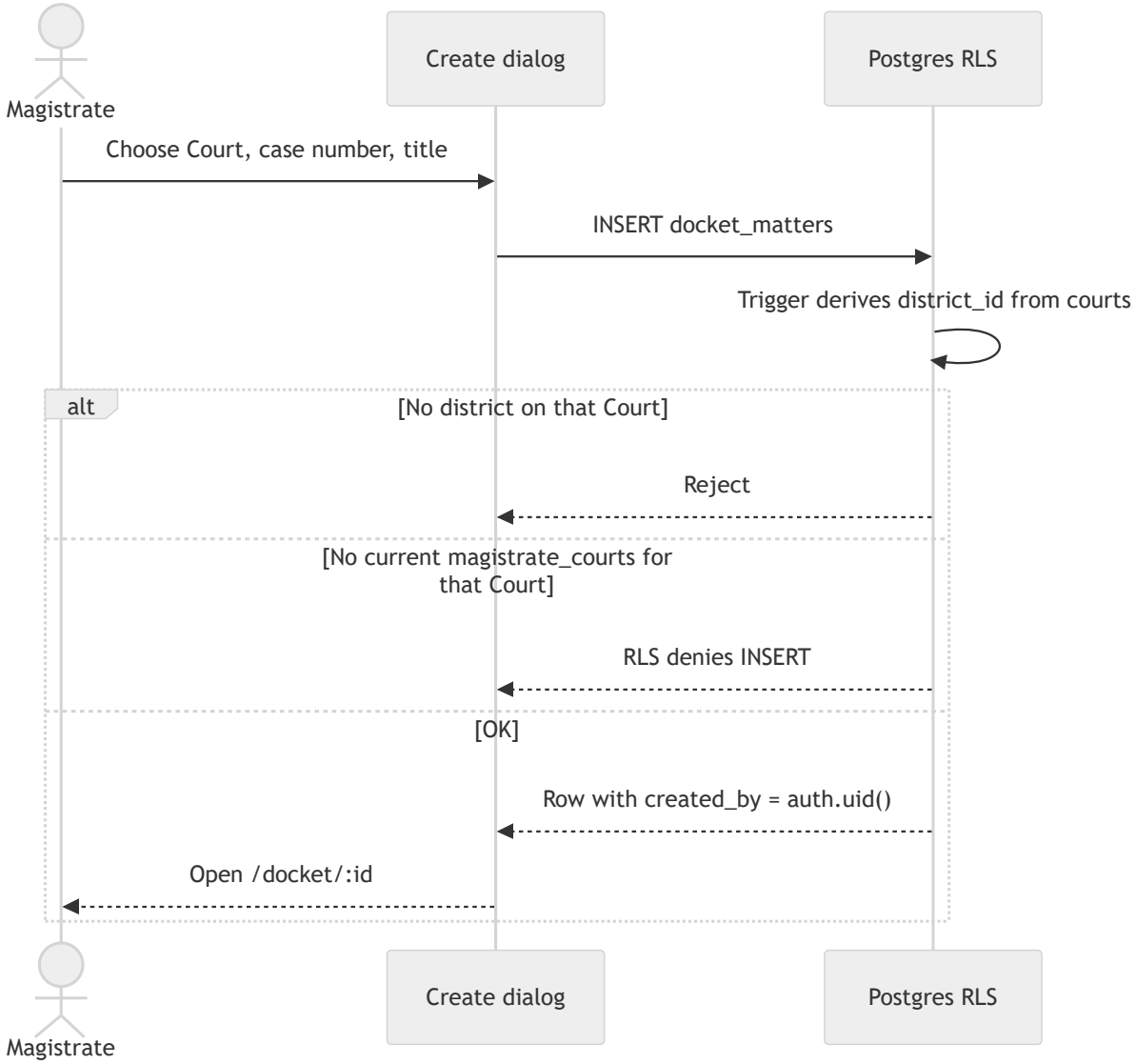
Screen map

Path	Who	Purpose
`/dashboard`	Any signed-in user	Courts, upcoming appearances, retained, drafts
`/docket`	Any	Matters via the three paths
`/docket/:id`	Lawful access	Operational workspace
`/judgments`	Any	Own + discoverable
`/case-law`	Any	Published canonical + own/discoverable personal
`/legislation`	Any	Published statutes + provision reader
`/quick-codes`	Any	Own snippets only
`/bench-notes`	Any	Own notes only
`/bookmarks`	Any	Own bookmarks
`/search`	Any	RLS-filtered global search
`/admin/court-assignments`	Admin UI role	Roster
`/admin/legal-library`	Admin UI role	Ingestion + review

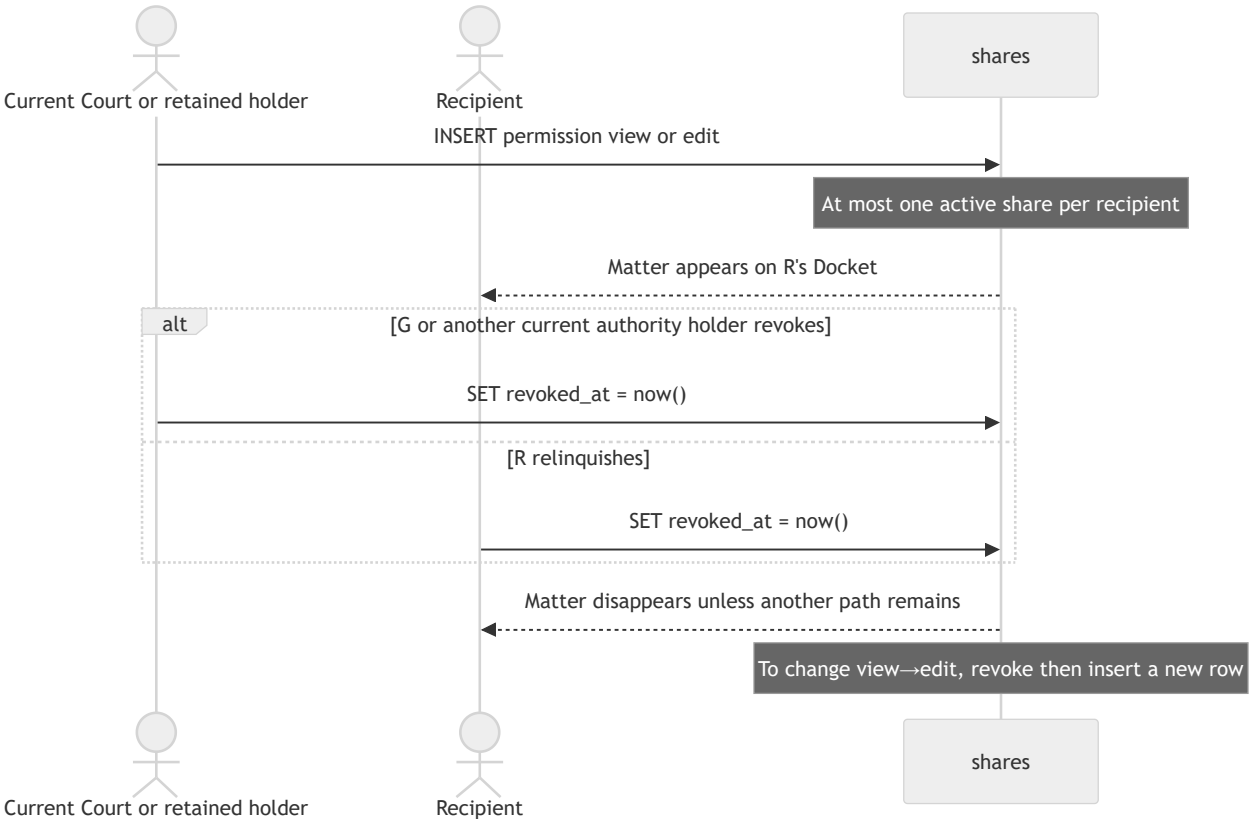
10. Sequence and state workflows

Time-ordered views of the rules in access control.

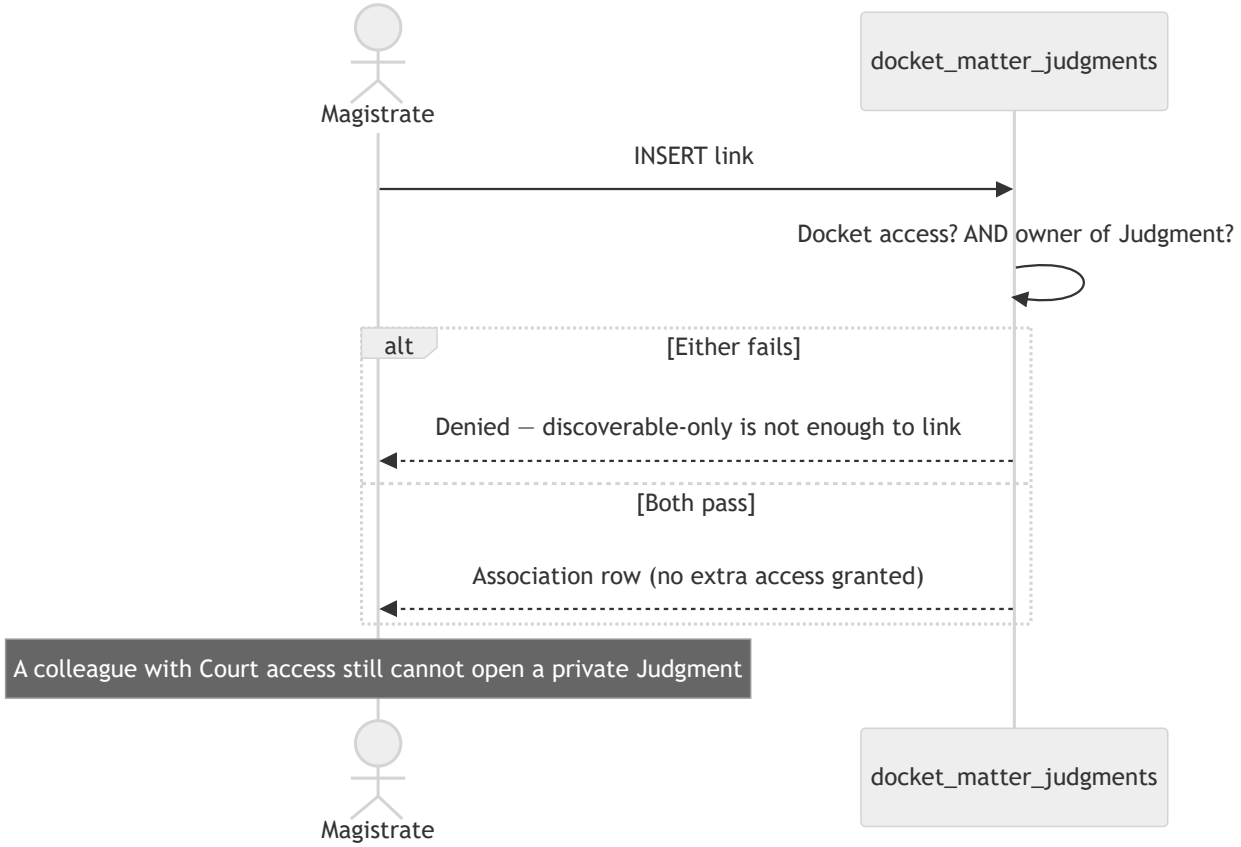
Create a Docket Matter



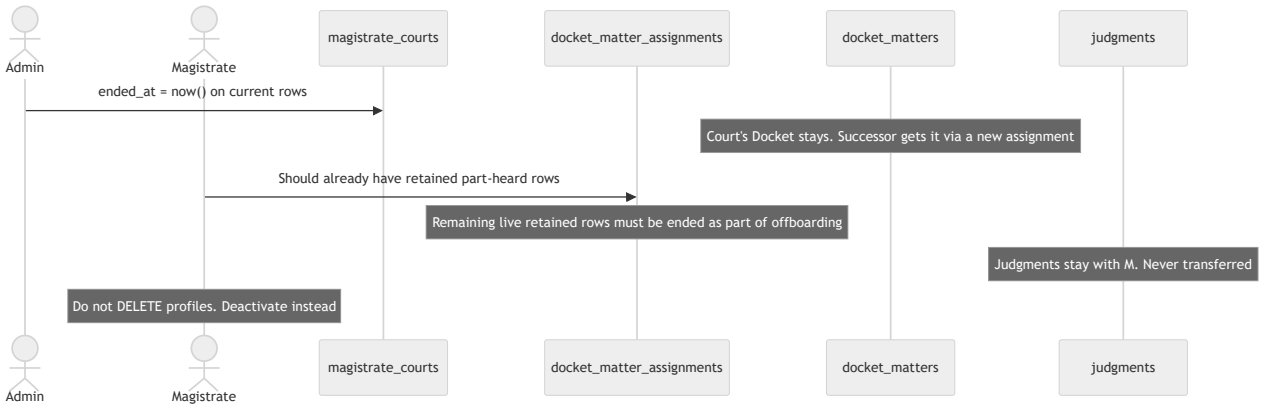
Share, then revoke



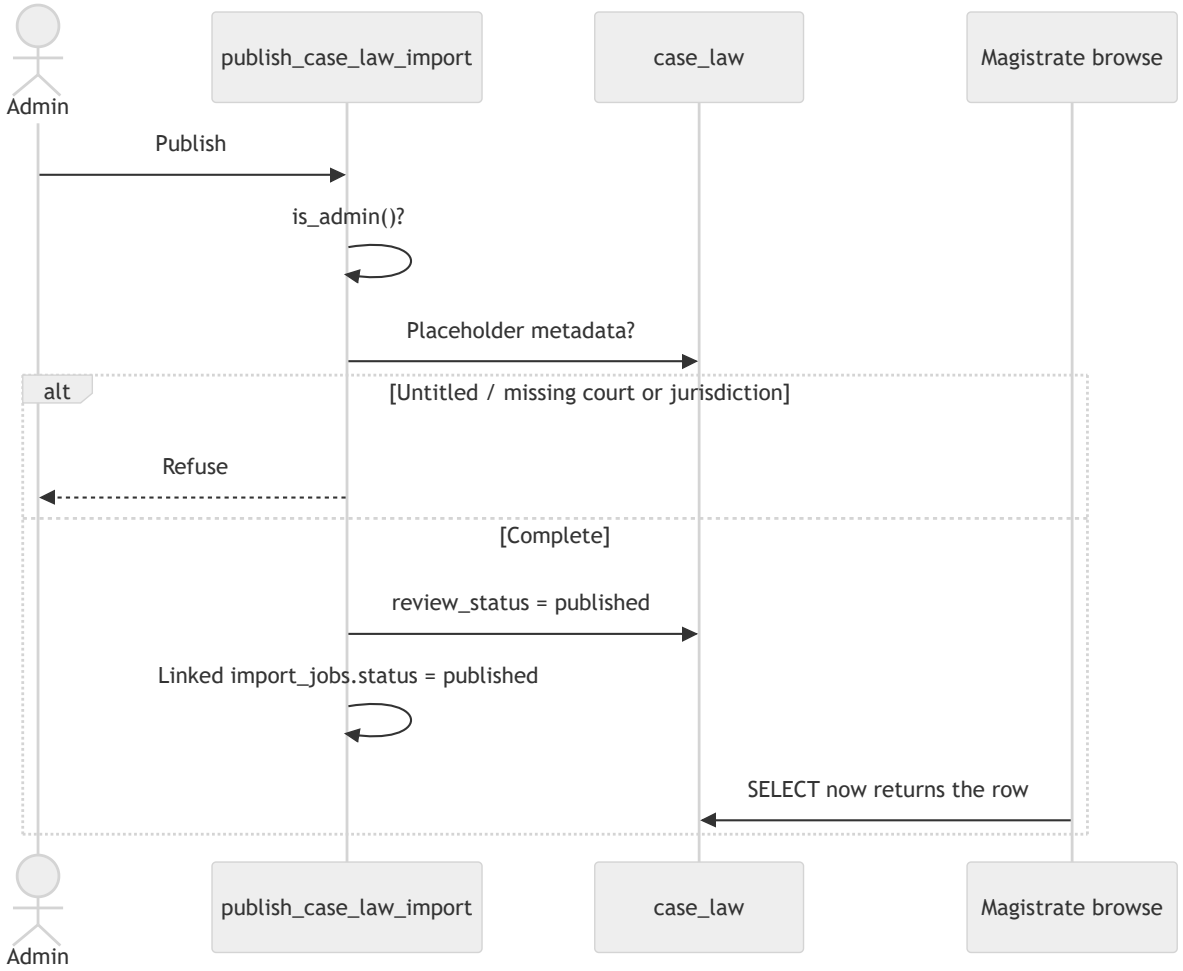
Link a Judgment to a matter



Offboarding a sitting magistrate



Publish canonical Case Law

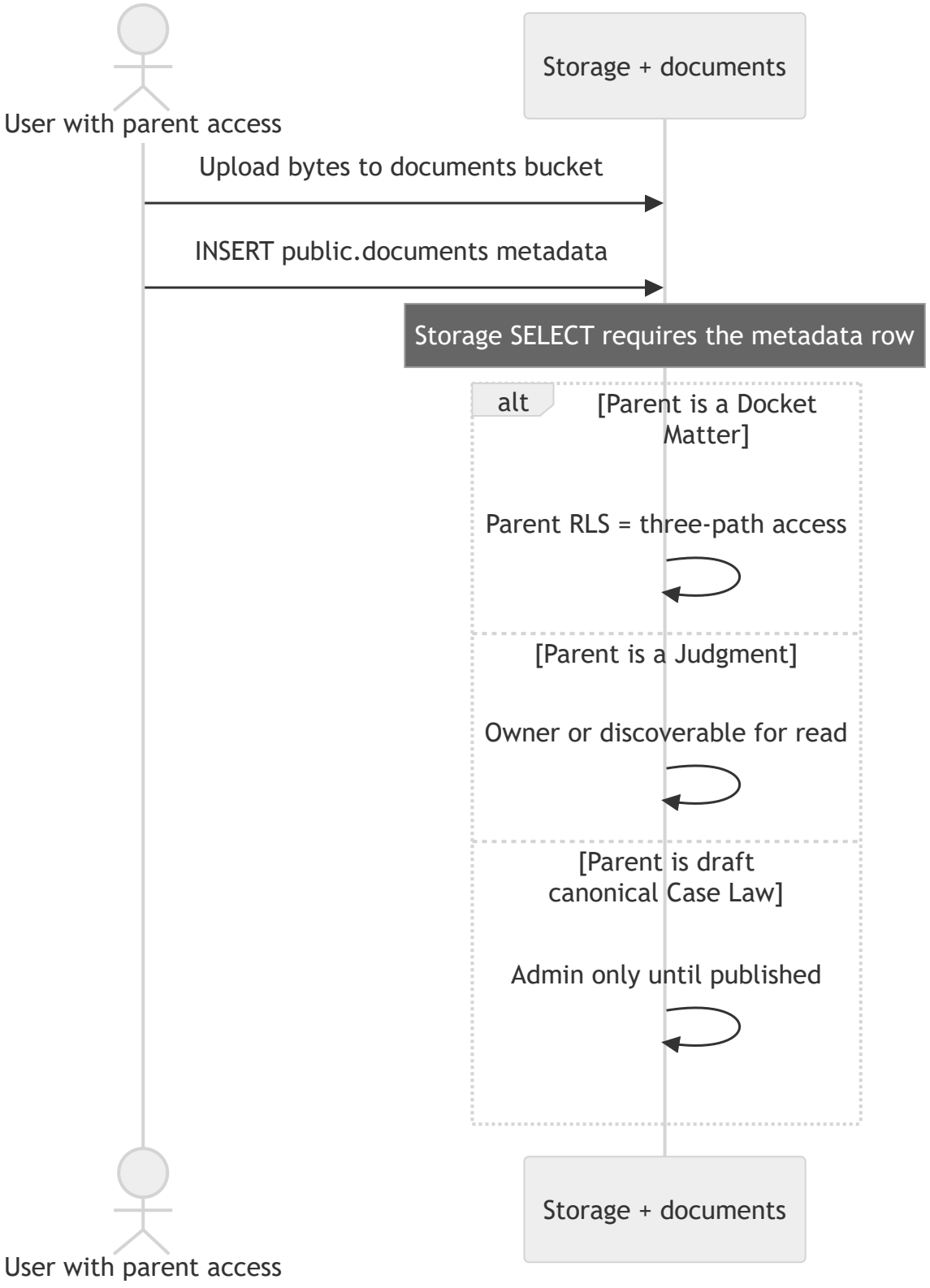


Quick Codes (as implemented)

Quick Codes are private text snippets (`code_word` unique per owner). The live workspace **Copy** action puts `content` on the clipboard. There is no in-editor expander. Association tables exist; the SPA associations dialog is read-only.

Nobody else ever reads another user's Quick Codes — not even an administrator, and not via a Docket share.

Document upload



What is not a workflow yet

Topic	Status
Outlook calendar sync	Columns reserved; no sync
Judgment / personal Case Law sharing	Designed in principle; `shares.item_type` is Docket-only in 0037
Clerk-specific permissions	Role exists; no Clerk RLS
URL crawlers / AI extract	Not built
Scanned-PDF OCR	Built in-browser (Tesseract.js); curator must still verify
Password reset completion page	Email currently returns the user to `/login`
Profile / Settings	Disabled in the user menu
View-share greying out editors	RLS only; controls stay enabled